

סיכום הרצאות 2026 – מערכות הפעלה

מרצה: פרופסור דוד סרנה

© גיא יער-און

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

תוכן עניינים:

2	הרצאה 1: חזרה על מבנה מחשב, מבוא למערכות הפעלה
9	הרצאה 2: ניהול זיכרון, מבנה מערכת ההפעלה, system calls
16	הרצאה 3: מכונה וירטואלית, תהליכים (Process)
22	הרצאה 4: המשך תהליכים, Sockets, Threads

הרצאה 1

מהי מטרת הקורס? עד היום התעסקנו ברמת תוכנית בודד ותהליך בודד: סינטקס של קוד, אלגוריתמים בהם משתמשים, כתיבת קוד נכון ויעיל וכו'. בקורס – אנחנו נתמקד בעולם בו פועל התהליך שלנו והאינטראקציה של אותו תהליך עם הסביבה, וכמובן נדון בכמה וכמה תהליכים. במהלך הקוד נדון בכמה וכמה שאלות:

- מה הן מערכות הפעלה?
- מה הן יודעות לעשות?
- כיצד מערכות הפעלה בנויות?
- מרכיבים עיקריים של מערכות הפעלה – תהליכים, Threads, CPU-Scheduling, וכו'.

נעיר מראש שבמהלך ההרצאות נדון בעקרונות כלליים של מערכות הפעלה – ולכן: בהרבה מהפעמים התשובה תהיה 'לא ניתן לדעת', שכן זה יהיה מאוד תלוי במערכת ההפעלה שתרוץ. **בתרגול:** נדון במערכת הפעלה ספציפית: Linux.

חזרה על מבנה מחשב:

- ארכיטקטורת Van-Neumann: משמשת כבסיס לרוב מערכות המחשבים המודרניות, היא מאוד מאוד פופולרית היום בעקבות יתרונותיה – אנחנו שומרים את הקוד ואת הdata שלנו באותו מקום בזיכרון, מה שהופך את התכנון והיישום של תוכנה וחומרה לפשוטים יותר. כמו כן, הארכיטקטורה נתמכת ע"י רוב שפות התכנות והכלים המודרניים. עם הזמן – התגלו בשנים האחרונות מגבלות בארכיטקטורה זו.
מגבלותיה במערכות מודרניות:
- 1. מחשוב מקבילי – מתאימה יותר למחשוב סדרתי, בעוד מערכות מודרניות דורשות יותר יכולת של תכנות מקבילי עם כמה וכמה מעבדים בו זמנית.
- 2. אבטחת מידע – העובדה שהקוד והנתונים נשמרים באותו מקום בזיכרון הוא פתח עבור "פולשים" שיוכלו לגנוב לי קוד.

במערכת ישנם 4 מודולים מרכזיים:

1. Memory – זיכרון: מאחסן את כל ההוראות ואת כל הdata של התוכנית שלנו. יש לנו בו את Control-Unit שאחראי על הרצת התוכנית שלנו ואת הAlu: אחראי על פעולות חשבון ולוגיקה לפי דרישות התוכנית וכן יש לנו בו את הIO לתקשורת עם "העולם החיצון".
- הזיכרון הוא אוסף של הרבה מאוד תאי זיכרון, כאשר גודל כל תא הוא קבוע. יחידת הזיכרון הבסיסית היא bit ותא זיכרון לרוב הוא byte בודד. הוא יקר ביחס לדיסקים וכדומה.
- הזיכרון עליו אנחנו מדברים נקרא RAM (Random Access Memory): מדובר בזמן גישה זהה לכל תא זיכרון. מדוע זה טוב לנו? כי זה גורר שלא אכפת לנו היכן התוכנית תשב בזיכרון – זמן הגישה זהה.
- WORD:** אוסף כל כמה ביטים (בהתאם לארכיטקטורה של המעבד), מדובר ביחידת המידע שמועבדת בפעולה בודדת. זהו גודל שלרוב זהה לגודל הרגיסטרים ולBUS. לרוב מדובר ב8 ביטים.
- לכל תא זיכרון ישנה כתובת:** את הכתובת אנחנו צריכים על מנת שנדע היכן הנתון נמצא בזיכרון. את הכתובות מייצגים באמצעות מחרוזת של ביטים. בהינתן n ביטים, נוכל לתמוך בזיכרון בגודל של $2^n - 1$ כתובות. מדובר בכמות המקסימלית של זיכרון שנוכל לשים במערכת שלי, כיוון שלא נוכל לייצג עוד זיכרון בשיטה זו.

סיכום מערכות הפעלה | © גיא יער-און

- את גודל הזיכרון מייצגים בבייטים – KB, MB, GB, TB וכו'.
- Hz הוא מס' הגישות לזיכרון שאנחנו יכולים לעשות בשנייה. כלומר 1 Hz הוא מחזור (Cycle) אחד בשנייה. מהו סייקל? יחידת העבודה הקטנה ביותר של זיכרון. הזיכרון יכול לבצע פעולה בכל רגע נתון. ככל שמס' הסייקלים גדול יותר פר שנייה הזיכרון עובד מהר יותר כי הוא מספיק לבצע יותר פעולות.
- מהירות הוא הנתון שאומר כמה סייקלים הזיכרון מבצע בשנייה אחת.

במקום לענות על השאלה: "כמה סייקלים יש בשנייה", נוכל לחשב כמה שניות לוקח סייקל אחד בודד:

$$\text{Time Per Cycle} = 1/\text{Hz}$$

לצורך ההמחשה: אם משהו קורה פעמיים בשנייה, כלומר 2 Hz, אז די ברור שכל פעם כזו (סייקל) לוקח חצי שנייה. כלומר, $\frac{1}{2}$.

פעולות על הזיכרון:

מערכת הזיכרון מופעלת באמצעות:

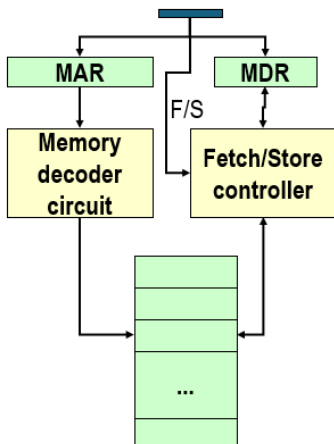
1. רגיסטר כתובות זיכרון – MAR – בעת שליפה ישמש לקרוא את הערך שאנחנו קוראים מהזיכרון.
2. רגיסטר נתוני זיכרון – MDR – ישמש לקרוא את הערך שנרצה לכתוב בזיכרון.
3. אות לשליפה/אחסון – אות חשמלי על מנת להבין האם הפעולה הבאה תהיה שליפה או אחסון.

- **שליפה – Fetch:** שליפת עותק של תוכן בתא הזיכרון עבור הכתובת שצוינה. מדובר בפעולה לא הרסנית כי היא משמרת את הערך בתא הזיכרון.

1. טוענת את הכתובת לתוך MAR

2. מבצע Decode לכתובת הזו: יחידת הזיכרון מפענחת את הכתובת שבMAR כדי לאתר פיזית את התא המתאים בזיכרון, היא תעתיק את התוכן של תא הזיכרון לתוך MDR
- אחסון – Store:** אחסון הערך שצוין לתוך תא הזיכרון עבור הכתובת שצוינה. פעולה הרסנית, דורסת את הערך הקודם בתא הזיכרון.

1. טוען את הכתובת לתוך MAR: המעבד מציב שם את כתובת התא אליו הוא רוצה לכתוב.
2. טוען את הערך לתוך MDR: המעבד מציב שם את הערך החדש שהוא רוצה לאחסן.
3. מבצע Decode של הכתובת לMAR: יחידת הזיכרון מפענחת את הכתובת בMAR על מנת לדעת להיכן לכתוב.
4. מעתיק את התוכן של MDR לתוך תא זיכרון עם הכתובת הספציפית שנבחר וכך מחליף את המידע הישן.



מודול Input/Output (IO):

מטפל בהתקנים שמאפשרים למערכת המחשב לתקשר וליצור אינטראקציה עם העולם החיצון – לתקשר וליצור אינטראקציה עם העולם החיצון (מסך, מקלדת), לאחסן מידע.

לכל התקן של IO ישנו התקן בשם **IO CONTROLLER**: הבעיה היא שמהירות התקני IO נמוכה מאוד ביחס למהירות הRAM – ולכן בכל שימוש בIO נעכב את כל המערכת, לכן הפתרון הוא שימוש בהתקן זה. מדובר במעבד ייעודי שכולל **רכיב זיכרון קטן (IO Buffer)**: יש בו לוגיקת בקרה לניהול התקן הקלט/פלט: הוא ינהל עבורנו בצורה עצמאית את האינטראקציה עם המכשיר עצמו. למשל: הזזת זרוע הדיסק. היתרון: כיוון שהוא עצמאי, אפשר לבצע את הפעולות שלו במקביל למעבד שלי שמבצע פעולות של התוכנית. בסיום ניהול האינטראקציה

סיכום מערכות הפעלה | © גיא יער-און

עם device, ה-CONTROLLER יודיע לי על כך. ישנם כמה וכמה Controller's: אחד ייחודי לכל סוג של מכשיר. למשל – ישנו Device Controller עבור המסך שיכול לנהל כמה וכמה מסכים.

מודל ה-ALU:

מיועד לבצע פעולות מתמטיות, לוגיות. במחשבים של היום ה-ALU משולב כבר בתוך ה-CPU. הוא מורכב ממעגלים לביצוע הפעולות האריתמטיות/לוגיות, רגיסטרים לאחסון תוצאות חישוב הביניים, BUS שמחבר בין השניים.

מודל ה-Control Unit (יחידת הבקרה):

- התוכנית מאוחסנת בזיכרון כפקודות בשפת מכונה (לאחר שהקומפיילר הפך אותם לפקודות בשפת מכונה).
- תפקידה של יחידת הבקרה הוא לבצע תוכניות על ידי חזרה על השלבים הבאים:
 1. שליפה – Fetch: שליפת הפקודה הבאה לביצוע מהזיכרון
 2. פענוח – Decode: קביעת הפעולה שיש לבצע
 3. ביצוע – Execute: ביצוע הפעולה ע"י שליחת אותות מתאימים ל-ALU, לזיכרון ולתתי המערכות של הקלט/פלט. **תמיד שלב זה מתבצע מבחינת הקורס שלנו.**

הוא ממשיך לבצע את 3 הפעולות הללו, אחת אחרי השנייה: עד לפקודת עצירה.

פקודות בשפת מכונה:

פקודה בשפת מכונה מורכבת מ:

1. קוד פעולה: מציין איזו פקודה עלינו לבצע.
2. שדות כתובת – מציינים את כתובות הזיכרון של הערכים שעליהם הפקודה שלנו תפעל.

Opcode (8 bits)	Address 1 (16 bits)	Address 2 (16 bits)
00001001	0000000001100011	0000000001100100
קוד פעולה 9	כתובת זיכרון 99	כתובת זיכרון 100

לדוגמה: add x,y: אנחנו צריכים לקרוא את כתובת הזיכרון של x ואז את הכתובת של y וכן יש את ה-OPCODE של הפקודה. וזה יראה כך:

PC (Program counter) – רגיסטר ששומר את הכתובת של הפקודה הבאה בתור לביצוע. בעת שפקודה נשלפת מהזיכרון, הוא מתעדכן כדי להצביע על הפקודה הבאה בזיכרון.

IR (Instruction Register) – רגיסטר שמחזיק את הפקודה עצמה שנשלפה כרגע מהזיכרון, שומר את ה-OPCODE שעשינו לו Fetch מתוך הזיכרון.

Instruction Decoder: מפענח הפקודות. יחידה לוגית שמקבלת את הפקודה מה-IR ומפענחת אותה. הוא מתרגם את הקוד הבינארי של הפקודה לאותות חשמליים שמפעילים את הרכיבים הרלוונטיים. הוא הרכיב החומרתי שמופעל בעת ביצוע Decode לפקודה.

מבנה מערכת המחשב:

ניתן לחלק את מערכת המחשב לארבעה מרכיבים:

- אפליקציות (Applications): מגדירים כיצד יבוצע השימוש במשאבי המערכת על מנת לפתור בעיות חישוביות של משתמשים. היא יכולה להיות קומפיילר, משחק וידאו, אתר וכו'.
- משתמשים: לא בהכרח מדובר באנשים – גם מחשבים מרוחקים הם משתמשים. למשל עבור web server ישנם מחשבים שפונים אליו כבקשות ומוגדרים כמשתמשים.

סיכום מערכות הפעלה | © גיא יער-און

- מערכת ההפעלה: נמצאת מעל החומרה. היא שולטת על ומתאמת את השימוש במשאבי החומרה בין התהליכים והמשאבים השונים. מדובר על מערכות שיש בהן הרבה מאוד תהליכים שרצים – ואיננו יודעים מי מבין התהליכים יותר חשוב או יקר. עלינו רק לשלוט ולנהל אותם.
- חומרה (Hardware): ה"ברזלים". אילו משאבי מחשב בסיסיים יש לנו? CPU, Memory, IO Devices וכו'.

הגדרה: מערכת הפעלה היא תכנית אשר משמשת כחוצץ בין המשתמש של מערכת המחשב והחומרה של מערכת המחשב. מערכת ההפעלה נמצאת במכונות, במכשירים ביתיים, בטלפונים, מחשבים אישיים ותעשייתיים וכו'. מטרותיה: הרצת תוכניות המשתמש בצורה קלה, הפיכת השימוש במחשב לנוח יותר ושימוש יעיל יותר בחומרת המחשב.

נשים לב: מערכת ההפעלה מוגדרת כתוכנית גם כן – ולכן דורשת משאבים גם בעצמה. כלומר: גם כשלכאורה המחשב נראה שלא עושה כלום, המעבד מריץ קוד של מערכת ההפעלה.

שאלה. האם יכולה להיות מערכת מחשב ללא מערכת הפעלה? (כלומר, אם אפשר לקנות מחשב ולא להתקין עליו מערכת הפעלה).

תשובה. בתכלס, כן, אבל. זה ייצור עבורנו הרבה מאוד בעיות:

- נצטרך לכתוב תוכניות בקוד מכונה.
- לא נוכל להריץ יותר מתוכנית אחת בו זמנית.
- נצטרך לטפל בעצמנו בכל המקרים והתגובות – שגיאות, תזוזת עכבר, לחיצה על מקשים וכו'.
- נצטרך לנהל אינטראקציה עם החומרה בעצמנו.

מתי כן נראה את זה? במערכת סגורה – שבה אין אינטראקציה עם האדם, שהופך את התנהגות התוכנית ללא דטרמיניסטית. הרי, אם המערכת סגורה וידועה והכל בה דטרמיניסטי אז אין לנו צורך במערכת הפעלה (שדורשת הרבה משאבים).

סוגי קטגוריות מחשוב:

- **Supercomputer:** מחשב על. מחשב נמצא בחזית יכולת החישוב, במיוחד במהירות החישוב. מחשב מאוד יקר אותו מדינות/אוניברסיטאות קונות.
- **Mainframe:** מחשבים חזקים (מבחינת אמינות, זמינות וביצועים). משמשים בעיקר ארגונים גדולים ליישומים חשובים – כמו רישום אוכלוסייה, ניהול מלאי וכדומה. מהירות פחות גבוהה משל מחשב העל, אך עדיין גבוהה.
- **Personal Computer (PC):** כל מחשב רב תכליתי שגודלו, יכולותיו ומחירו המקורי הופכים אותו לשימושי עבור משתמשים פרטיים. אתה מוכר אותו עבור אדם ואינך יודע מה הוא יעשה בו – אם הוא ישתמש בו עבור מוזיקה בנטפליקס, או שיכתוב בו קוד.
- **Handheld:** דומה ל-PC, מכשיר מחשב אך עם סוללה שניתן לתפעול ביד אחת. הוא כולל בדרך כלל מסך תצוגה עם קלט של מגע. (כמו טלפון סלולרי).

לכל מערכת הפעלה יש מטרות ועיצוב שונים. למשל:

- **Mainframe:** הפוקוס של מערכת ההפעלה יהיה על ניצול מקסימלי של משאבי חומרה – פחות נוחות המשתמש, יותר ניצול מקסימלי של החומרה.
- **PC:** תמיכה מקסימלית בהרצת תוכניות של המשתמש – נוח עבור המשתמש.

UI-User Interface: המעטפת הוויזואלית והאינטראקטיבית שדרכה המשתמש מתקשר עם המכונה (כמו כפתורים, תפריטים ועיצוב המסך).

סיכום מערכות הפעלה | © גיא יער-און

- **Handheld**: שיהיה נוח להריץ אפליקציות, כי אין מקלדת/עכבר. נרצה גם ביצועים טובים פר יחידת ניצול של הסוללה.

בבירור – הנוחות מגיעה על חשבון היעילות. כמו תמיד, נאלץ לוותר על חלק מהיעילות עבור הנוחות ולוותר קצת על הנוחות עבור היעילות.

תפקידיה של מערכת ההפעלה:

- **Resource allocator**: בדומה לממשלה, שמקצה כמה מהתקציב ילך לכל תחום (חינוך/בריאות וכו'), מערכת ההפעלה מנהלת את המשאבים, היא מחליטה במצבים של קונפליקט על מנת שהשימוש במשאבים יהיה יעיל והוגן.
- **Control program**: שולטת על ההרצה של התוכניות השונות על מנת למנוע שגיאות, ולמנוע שימוש מוטעה במערכות המחשב.

תפקידיה חשובים מאוד בעיקר כאשר ישנם כמה משתמשים שמחברים לאותו mainframe.

שאלה. האם צייר, סוליטייר וכו' נחשבים כחלק ממערכת ההפעלה?

תשובה. אין תשובה אחת. יש כאלו שסוברים שכן – כל מה שהגיע בהתחלה כחלק מדיסק ההתקנה של מערכת ההפעלה הוא חלק ממערכת ההפעלה, ויש כאלו שסוברים שלא, והם לא חלק ממערכת ההפעלה.

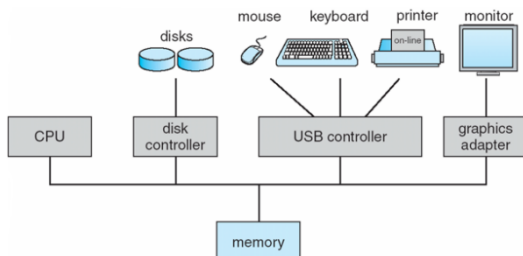
הגדרה: התוכנית/תהליך שרץ כל הזמן במערכת המחשב נקרא kernel. כל שאר התהליכים שרצים ע"פ דרישה מקוטלגים כ system programs (תוכניות שהגיעו יחד עם מערכת ההפעלה). למשל – הצייר מקוטלג כך. הוא הגיע עם מערכת ההפעלה, אך לא רץ כל הזמן. לעומת זאת application programs אלו דברים כמו chrome, הורדנו למחשב אך זה לא הגיע עם מערכת ההפעלה. כאשר אפליקציה רוצה לקרוא קובץ, להפעיל מצלמה, להקצות זיכרון – kernel מטפל בזה. הקרנל נותן יכולות בסיסיות, ותוכניות המערכת הופכות אותן למשהו נוח לשימוש.

Middleware frameworks: מדובר בשכבת ביניים שמטרתה היא לתת למפתחי אפליקציות שירותים מוכנים מבלי שיצטרכו להתעסק ישירות בפרטים הנמוכים של הקרנל. זה עדיין יעבור פונקציונאליות של הקרנל.

הפעלת המחשב: נניח וקנינו מחשב חדש, עליו כבר מותקנת מערכת ההפעלה. לחצנו על כפתור ההדלקה. מה קורה? ברגע שהדלקנו את המחשב ישנו תהליך של reboot. עולה **reboot program**: מדובר בתוכנית דטרמיניסטית שסגורה על עצמה ויודעת להתקדם קדימה ללא עזרה מהמשתמש – היא מאוחסנת ב-EEPROM (רכיב חשמלי, לא נמחק בניגוד ל RAM, בעת שנסגור את המחשב ונדליק יום למחרת הוא יישאר שם). תוכנית זו מאתחלת את כל מרכיבי המערכת – תוכן הזיכרון, הרגיסטרים של המעבד וכו'. בנוסף: היא זו שמעלה את kernel של מערכת ההפעלה ומעבירה אליה את השליטה. בנקודה זו – מערכת ההפעלה הופכת לעצמאית, כל הקוד שלה טעון ומרכיביה מאותחלים. ואז – מערכת ההפעלה לא עושה כלום: היא מחכה, עד שיהיה אירוע כלשהו: שהמשתמש יפעיל את העכבר, יזיז משהו וכו'.

מעבדים:

מערכת ההפעלה מאופיינת ע"י מעבד (אחד או יותר, כיום יש כמה וכמה) Device 1 controllers: כמו Disk controllers controller וכו'. כל אחד מהם אחראי על סוג מסוים של devices. הם מחוברים באמצעות common bus שמאפשר להם גישה לזיכרון משותף, כך:



כל device controller אחראי על סוג מסוים של device. המחשב לא יודע כיצד לדבר ישירות עם המנוע של הארד דיסק, ולכן הוא מדבר עם הבקר והבקר יודע כיצד לנהל את המכשיר הספציפי שמוטאם לו.

סיכום מערכות הפעלה | © גיא יער-און

המידע מ ואל device מנוהל באמצעות Local Buffer: המעבד מעביר מידע מ/אל הזיכרון הראשי אל הלוקאל באפר. כיוון שהתקני IO איטיים מאוד בהשוואה למעבד, המידע לא יעבור ישירות מהדיסק למעבד. הלוקאל באפר הוא זיכרון מקומי קטן. המידע מהמכשיר נאסף קודם כל בבאפר של הבקר, כשהמידע מוכן הוא מועבר אל הזיכרון הראשי, ומשם למעבד (ולהפך). במקום שהמעבד ישב וישאל כל הזמן "סיימת? סיימת?" – הוא בונה על כך שבעת שהdevice controller יסיים הוא ישלח interrupt: לכן הוא יכול להמשיך לבצע פעולות אחרות תוך כדי.

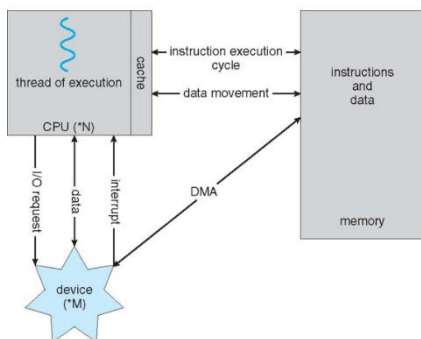
ברוב המערכות נמצא לפחות מעבד אחד מסוג **general purpose**: מעבד שיועד לעשות הכל, הוא לא נבנה בצורה שנותנת יתרון ליישום מסוים – הוא נבנה בצורה כללית ויועד לבצע את הכל. לצד מעבד זה, מוסיפים מעבדים נוספים מסוג **special purpose**: למען מטרה מסוימת. למשל – GPU (מעבד עבור גרפיקה). לצד המעבדים הנוספים, תמיד נהיה חייבים את המעבד הכללי.

מערכות **Multiprocessors** הן מערכות בהן יש יותר ממעבד אחד – ישנה תקשורת בין המעבדים, שחולקים את אותו bus ולעיתים גם את הזיכרון. מה היתרונות של מערכת שכזו?

- מהירות, כמובן.
 - כלכלי: אך למה שלא נקח מעבד פי 2 יותר מהיר, במקום לקחת 2 מעבדים? זה מגיע כמובן מבחינה כלכלית – לרוב ליצור מעבד מהיר פי 2, הרבה יותר יקר מאשר ליצור 2 מעבדים במהירות שקולה.
 - בעת שיש תקלה, אם יש לנו רק מעבד אחד תקלה במעבד משביתה את המערכת כולה. בעת שיש כמה מעבדים, הם יכולים לכפות על המעבד שהתקלקל – המהירות תרד, אך המערכת לא תושבת.
- ישנן שתי ארכיטקטורות של ריבוי מעבדים:

- **Asymmetric Multiprocessing**: משימות מסוימות מוקצות למעבדים מסוימים בלבד. בפרט, יתכן כי מעבד יחיד יהיה אחראי על טיפול בכל interrupts במערכת ואולי אף יבצע פעולות קלט/פלט.
 - **Symmetric Multiprocessing**: מתייחס לכל רכיבי העיבוד במערכת באופן זהה – אפשר לפנות לכל המעבדים בעת דרישה.
- בכל פעם שנאמר בקורס core: הכוונה למעבד. (זה לא כך בפועל, אך זה הדיון במסגרת הקורס).

- **Clustered systems**: דומה לריבוי מעבדים, אך בריבוי מעבדים ישנם כמה מעבדים שחולקים אותו לוח אם ואותו זיכרון: כאן מדובר במערכות נפרדות (בזיכרון, לוח אם) שעובדות יחד. הן חולקות אחסון משותף דרך רשת שנקראת SAN. היתרון הגדול הוא הזמינות – גם אם אחת המכונות קורסות, השירות לא יקרוס. בדומה, ישנן 2 שיטות לגישה זו:
1. **Asymmetric Clustering**: ישנה מכונה אחת שהיא במצב "Hot-StandBy": היא לא עושה כלום, והיא רק מחכה אם אחת השרתים יקרוס: היא תכנס במקומו.
 2. **Symmetric Clustering**: כל המכונות עובדות ומריצות אפליקציות במקביל, אם אחת נופלת הן מתחלקות בעומס שלה. זה יעיל יותר כמובן כי כל החומרה מנוצלת.



איך מחשב מודרני פועל? יש לו מעבד – שמדבר גם עם הזיכרון, וגם עם device. בזיכרון – יש לנו גם פקודות, וגם דטה. המעבד יהיה זה שייקח דטה מהזיכרון, ויעביר אותו אל device: באמצעות device controller.

אם יש בקשה של device מהמעבד לבצע פקודות IO הוא ישלח בקשה: IO REQUEST, לשם כך לפעמים יהיה צריך להעביר data בין CPU לdevice. בסיום – device ישלח interrupt, עליה נרחיב כעת:

Interrupt: פסיקה – מדובר באות חשמלי שמתקבל במעבד מרכיב החומרה (דיסק, עכבר וכו'). נוצר ע"י device. הוא מעביר את השליטה ל **interrupt service routine (ISR)**. זו הדרך של device לומר "סיימתי": זו הדרך שלנו לעצור את מה שעושה המעבד. המעבר שעצר, יטפל כעת בinterrupt וכשהוא יסיים נוכל לחזור לעשות מה שעשינו קודם. הוא מחייב שמירה של ה כתובת של ה interrupted instruction ושל תוכן הרגיסטרים (על מנת שנוכל לחזור בהמשך למצב שבו היינו – עם ערכי הרגיסטרים שהיו לפני ביצוע ה interrupt). להכל יהיה interrupt: בכל פעם שה device controller יסיים משהו – הוא ישלח. נבחין כי בעת לחיצה של מקש במקלדת – ה device controller יטפל בכל התהליך – יזהה איזה מקש במקלדת הוקש וכו', ורק בסוף הוא יבצע interrupt.

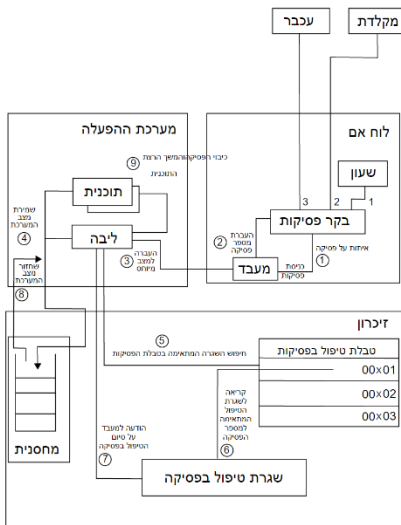
אם נסכם את התהליך, נעזר בתמונה משמאל להבנת התהליך:

שלב א': זיהוי – בלוח האם: התקן חיצוני (מקלדת/עכבר) שולח אות חשמלי לבקר הפסיקות. הבקר מעביר את האות אל המעבד. בקר הפסיקות שולח למעבד את ה"זהות" של הפסיקה (מספר הפסיקה. למשל: "פסיקה מספר 2 הגיעה דרך המקלדת")

שלב ב': החלפת הקשר – במערכת ההפעלה: המעבד מפסיק את ריצת התוכנית הרגילה ועובר ל kernel mode, על מנת שיוכל לגשת למשאבי מערכת מוגנים. כמו כן, המערכת שומרת את הנתונים הנוכחיים של התהליך (רגיסטרים, כתובות וכו') בתוך המחסנית על מנת שנוכל לחזור אליהם אחר כך ללא איבוד מידע.

שלב ג': איתור וביצוע – בזיכרון: חיפוש בטבלת הפסיקות: המעבד נגש לטבלת וקטורי הפסיקות בזיכרון ומחפש את הכתובת של הקוד שמתאים למס' הפסיקה שהתקבל בשלב 2. המעבד קופץ לכתובת שבטבלה ומצא, ומריץ את שגרת הטיפול בפסיקה (ISR), זהו הקוד שבאמת מטפל בלחיצת המקלדת או בתזוזת העכבר.

שלב ד': חזרה לשגרה – הודעה על סיום, המערכת מסיימת ומעדכנת את המעבד שהטיפול הושלם. שחזור מצב המערכת – שולפת מהמחסנית את הנתונים ששמרה ומחזירה אותם לרגיסטרים. המעבר חוזר למצב User Mode וממשיך להריץ את התוכנית בדיוק מהנקודה שבה נעצרה.



הרצאה 2

Trap: דומה לinterrupt, הוא לא אות חשמלי אלא הודעה שמעביר קוד שלנו. הוא software-generated interrupt. פונקציונלית, הוא גורם למערכת ההפעלה להתעורר ולעשות משהו, לכן הוא דומה לinterrupt, הוא נגרם בעקבות שגיאה/בקשה של תוכנית של המשתמש.

Interrupt Handling: כאשר נקבל interrupt ישנן 2 גישות עיקריות כיצד לטפל בו, ראשית עלינו להבין מה סוג interrupt שקרה. הגישות:

- **Polling:** יצרנו מראש קוד גנרי שיודע לטפל בכל interrupt שיגיעו (יודע לתת את כל התגובות לכל המקרים). ואז – כשנקבל interrupt נריץ את הקוד הגנרי ונבין באמצעות הקוד הגנרי מה קרה.
- **Vectored interrupt system:** יחד עם interrupt נקבל מס' כלשהו (למשל: מס' 7 הוא הודעה של המדפסת, מס' 3 הוא הודעה מהשעון וכו'). באמצעות המס' הנ"ל אנחנו יכולים ללכת אל טבלה בזיכרון – להגיע אל המקום המתאים בטבלה, ושם נמצא את כתובת הקוד שצריך לטפל בinterrupt הנ"ל.

בבירור, לכל interrupt יהיה קטע קוד רלוונטי (routine), ששמור בזיכרון. מהם היתרונות/חסרונות של כל אחת מגישות הטיפול?

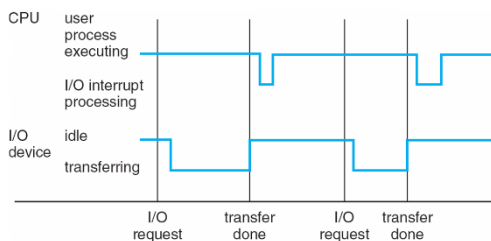
בגישת Vectored: נקבל שזמן הטיפול יהיה הרבה יותר מהיר – שכן נמצא את קוד הטיפול הרבה יותר מהיר (זו מעין "טבלת האש"), אך – כנראה שיש הרבה כפילויות: טיפול בinterrupt שהגיע מדיסק מסוג מסוים A וinterrupt שהגיע מדיסק מסוג B – יתכן חפיפה בקוד שלכאורה יכולנו לאחד אותם אם היה בידינו קוד מסוג Polling, ולכן אנחנו משתמשים בהרבה יותר זיכרון כי יש קודים נפרדים לכל אחד מהמקרים.

כיוון שהמהירות היא קריטית (המעבד עוצר, מחכה..) – רוב המערכות עובדות בגישת Vectored.

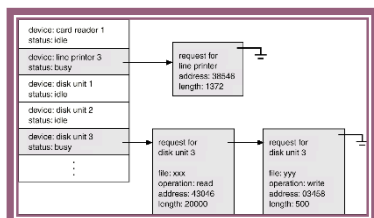
טיפול בIO:

סינכרוני: ברגע שהתוכנית של המשתמש מתחילה לרוץ, אנחנו מעבירים את השליטה לKernel שיפעיל את device controller ובאמצעותו את device עצמו, ועד שלא סיימנו את הטיפול בIO התוכנית של המשתמש לא תמשיך לרוץ

אסינכרוני: מבקשים את IO ממערכת ההפעלה באמצעות device controller ומיד מחזירים את השליטה לתוכנית של המשתמש – ברגע שהdevice יסתיים הוא ישלח interrupt. אך, נבחין כי יתכן ויהיה תלות להמשך קוד המשתמש בIO: ולכן במקרים אלו נעבוד בצורה סינכרונית. אם רק רוצים להדפיס למסך למשל – ואין תלות בהמשך הקוד בIO: נעבוד בגישה אסינכרונית.



משמאל – דוגמה למצב אסינכרוני: המשתמש רץ עד לקו השחור הראשון, שם המשתמש מבקש IO (למשל: לפתוח קובץ). במצב זה – מדובר בsystem call והשליטה עוברת למערכת ההפעלה – היא מריצה קוד בו היא מעבירה לdevice controller את המידע לרגיסטרים שהוא זקוק (זה החלק בין שני השחורים ש"נופל"), במקביל – כיוון שאנחנו באסינכרוני, תוכנית המשתמש ממשיכה לרוץ, עד לקו השחור ששם device controller סיים ושולח interrupt למעבד – ועובר להריץ קוד שמטפל באותו interrupt (זה המעבד) – זה ה"נפילה" בין הפס השחור השני לשלישי. לאחר מכן – התוכנית ממשיכה לרוץ (וקורים תהליכים דומים).



- מה קורה במצב אסינכרוני שבו לא סיימנו לטפל בבקשה שהגיעה, וכבר הגיעה עוד בקשה לdevice? לשם כך משתמשים במעין "תור", ישנה טבלה: Device-Status Table – כמו באיור משמאל. לכל device תהיה מעין שורה בטבלה, ובה הפניה ל"תור" בקשות.

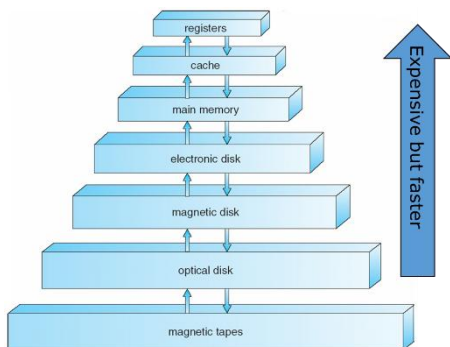
ניהול הזיכרון:

מדוע הזיכרון כל כך חשוב? instructions חייבות להיות בזיכרון על מנת שנוכל לבצע אותן, וגם data חייב להיות בזיכרון. ניהול זיכרון עוסק בהחלטה על מה יהיה/ישהה בזיכרון בכל רגע ורגע. מדוע שלא נשמור הכל בזיכרון בכל רגע נתון? כי הזיכרון ממש יקר – ומוגבל במקומו, ולא נוכל להחזיק הכל בזיכרון בכל רגע נתון.

תפקיד מערכת ההפעלה בנוגע לניהול הזיכרון:

- היא רושמת ועוקבת על החלקים בזיכרון שכרגע נמצאים בשימוש.
 - מחליטה על העברה של תוכן זיכרון שמשויך לתהליך מסוים – מ/אל הזיכרון: זה נקרא swapping. למשל: להעביר אל דיסק. מדוע שנעשה זאת? כי לפעמים אין מספיק מקום בזיכרון.
 - מקצה זיכרון לתהליכים, ולוקחת זיכרון מתהליכים, ויכולה לשפר זיכרון לתהליכים (לתת עוד זיכרון).
- Mass Storage Management:** פרט לזיכרון הראשי (אמצעי אחסון הגדול היחיד שהמעבד יכול לגשת אליו בצורה ישירה), ישנם דיסקים. מדוע שנשתמש בדיסקים?

- נרצה לאחסן נתונים שלא יכולים להכנס לRAM בגלל גודלם.
 - נרצה לאחסן נתונים שיהיו שם גם לתקופה ארוכה – גם אחרי שנכבה את המחשב.
- ניהול נכון של Mass storage (הדיסקים) משפיע מאוד על המהירות הכוללת של פעילות המחשב – כיוון שגישה לדיסק יקרה מאוד. תפקיד מערכת ההפעלה בהקשר: מנהלת את המקום הפנוי, מקצה מקום אחסון ומתזמנת פעולות דיסק.



משמאל: היררכיית Storage. ככל שנעלה בהיררכיה, התקן האחסון שנפגוש יהיה יותר מהיר, העלות שלו תהיה הרבה יותר יקרה פר כמות אחסון. משלב מסוים גם הזיכרון הופך לנדיף – הרגיסטרים והקאש נדיפים (נדיף הוא זיכרון שזקוק לזרם חשמלי רציף כדי לשמור על המידע שנמצא בתוכו. ברגע שמכבים את המחשב או שיש הפסקת חשמל – כל המידע שלא נשמר על הדיסק הקשיח נמחק לצמיתות).

שאלה. האם יתכן ואדם יגיע יום אחד – ויאמר שהצליח ליצור רמה בהיררכיה שנמצאת גבוהה יותר שהיא גם מהירה יותר וגם זולה יותר?

תשובה. כן: יכול להיות, אבל מה ההשלכות? היא תעלים מההיררכיה את כל מי שנמצא מתחתיה – כי לא נצטרך להשתמש בהם יותר. הרי היום משתמשים בדיסקים כי הם זולים יותר, למרות המהירות הלא טובה שלהם. אם נמצא זאת: לא נצטרך יותר להשתמש בכל מי שמתחתיו בהיררכיה. הערה: הגודל הפיזי של הרכיב – גם עשוי להיות שיקול (שכן אם יגיע אדם ויציע טכנולוגיה כזו – רק שהיא דורשת חדר שלם לאחסון, זה יהפוך לשיקול).

Caching: העתקת מידע למערכת אחסון מהירה יותר. למשל: אם נרצה לקחת מקובץ שנמצא בדיסק פעולת חישוב 2+3, זה יעבור מהדיסק לזיכרון הראשי, אל הקאש ואל הרגיסטרים.

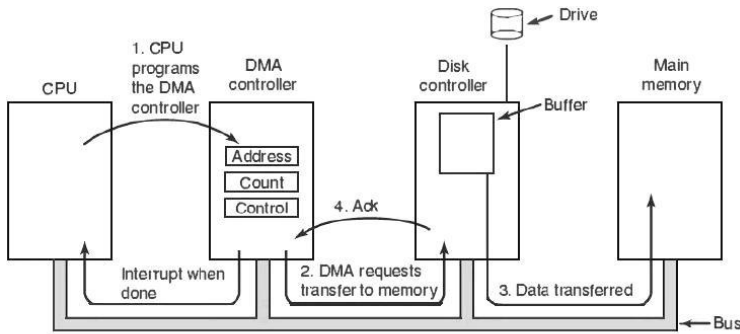


Figure 1. Operation of a DMA transfer.

נזכר בתמונה מעמוד 7 – הופיע שם DMA: כעת נסביר עליו.

המטרה של DMA היא "לעקוף" את המעבד. בשיטה הרגילה – המעבד צריך לקרוא כל בייט מהdevice ולכתוב אותו לזיכרון: מדובר בפעולה בזבזנית. כעת, dma מאפשר להתקן קלט/פלט להעביר בלוקים של מידע ישירות לתוך RAM מבלי שהמעבד יצטרך לגעת בנתונים בדרך.

בתמונה משמאל נראה תיאור של התהליך בו המידע עובר ישירות מהdevice אל הזיכרון הראשי.

שלב ראשון: המעבד לא רוצה להתעסק בהעברה של כל בייט ובייט – לכן הוא ניגש אל הDma controller ומטעין אותו בנתונים – כתובת (לאיזה כתובת בזיכרון לשפוך את המידע), כמה מידע להעביר, וcontrol: סוג הפעולה (קריאה/כתיבה).

שלב שני: הDma controller פונה לDisk controller ומבקש ממנו להתחיל להעביר נתונים.

שלב שלישי: בקר הדיסק שולף מידע מהDrive ושם אותו בבאפר: משם המידע עובר על הbus ישירות לזיכרון הראשי. המעבד לא רואה את המידע הזה עובר ולא נוגע בו – הנתונים נשלחים ישירות לזיכרון.

שלב רביעי: Ack – האישור: בכל פעם שבלוק מידע עובר בהצלחה מהבקר לזיכרון נשלח סיגל Ack חזרה לDma controller בשביל לעדכן את count (כמה נותר להעביר). כאשר count=0 הDma controller שולח Interrupt למעבד – רק עכשיו המעבד עוצר לרגע, מבין שהמידע מוכן בזיכרון הראשי ויכול לעבוד איתו.

Multiprogramming: היום, במחשב שלנו בבית אנחנו מריצים כמה תהליכים בו זמנית (למשל: פותחים סביבת פיתוח ומתכנתים, ותוך כדי וורד פתוח, ומאזינים לשיר בנגן וכו') – הרבה פעמים נוצר מצב בו כמות התהליכים גדולה מכמות המעבדים. זהו מצב של multiprogramming. כאשר יש תוכנית בודדת שרצה, אי אפשר לשמור על המעבד והתקנים השונים בנצילות מלאה. אנחנו מעוניינים לשפר את הנצילות של המעבד – ולכן אידיאלית נרצה שבכל רגע נתון תהיה תוכנית שרצה על המעבד. multiprogramming מאפשר כמות תהליכים גדולה מכמות המעבדים, בצורה של שיתוף: כשתהליך מסוים למשל פונה לIO או מסיים הרצה, מתפנה מקום לתוכנית אחרת. ולכן ישנו מעין תור של תהליכים – צריך לנהל זאת כמובן: אך בכל פעם תהליך אחר מקבל זמן מעבד.

Timesharing: עדיין מדובר בהרצה של כמות תהליכים שגדולה מכמות המעבדים, אך דואגים להחליף אותם על גבי המעבדים בתדירות כל כך גבוהה כך שכל התהליכים שהמשתמש מריץ ברגע זה רצים כרגיל – והוא לא רואה אף אחד מהם נתקע או מחכה שיתפנה זמן מעבד. תחושת המשתמש היא שכל התהליכים רצים ברגע נתון, גם אם הם לא באמת מקבלים כל שניה זמן מעבד. נדגיש כי:

- זמן התגובה חייב להיות פחות משנייה אחת.
- לכל משתמש יש לפחות תוכנית אחת בהרצה בזיכרון.
- אם לא ניתן להכיל את כל התהליכים בזיכרון – מתחיל swapping בין התהליכים.

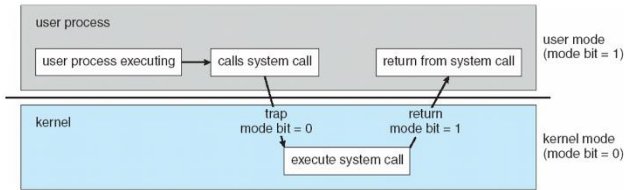
מעבר בין User mode לKernel mode:

מערכת ההפעלה צריכה לפעמים להריץ קוד שלה על מנת לבצע משהו עבור תוכנית של המשתמש – למשל בsystem call. הקוד של מערכת ההפעלה הוא יותר בטיחותי כי ישבו עליו הרבה ודיבגו אותו והוא לא ינסה לעשות דבר זדוני. היינו רוצים שהקוד הזה יעשה יותר. ישנן פקודות שנרצה להגדירן privileged: פקודות שרק מערכת ההפעלה יכולה לבצע – המשתמש לא.

סיכום מערכות הפעלה | © גיא יער-און

כיצד נדע האם אנחנו ב-User Mode? ישנו ביט מיוחד mode bit: אם ערכו 1 אנחנו ב-user mode, ואחרת אם הוא 0 אנחנו ב-kernel mode. אם אנחנו ב-user mode – לא נאפשר להריץ פקודות שהוגדרו כ-privileged.

בעת שנקרא ל-system call, נשלח trap למערכת ההפעלה וה-bit mode הופך להיות 0. מכאן: השליטה עוברת למערכת ההפעלה. בסיום, ה-bit mode הופך חזרה להיות 1: וחוזרים ל-user mode.



מבנה מערכת ההפעלה

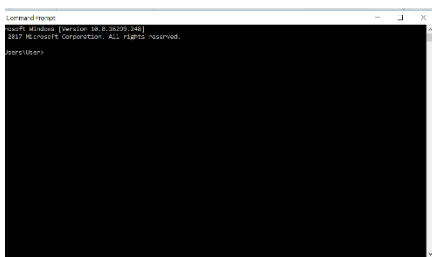
שאלה. מה הופך מערכת הפעלה להיות "מערכת הפעלה"? הרי – לכל מכשיר יש מעין ממשק למשתמש.

תשובה. איפה שיש ממשק משתמש בלבד – כמו בכפתורים במיקרוגל או במכונת הכביסה, במקומות כאלו הממשק הוא מעין שכבת תקשורת בין משתמש למכשיר. הממשק הנ"ל לא מייצר ניהול משאבים – כפי שנותנת מערכת ההפעלה. למעשה, מכשירים פשוטים יכולים לפעול עם תוכנה דטרמיניסטית פשוטה שמנהלת מס' פעולות מוגבלות מוגדרות בראש ואין צורך שיתעסקו בניהול משאבים. כלומר: למערכת הפעלה יש עושר בניהול משאבים ובמורכבות של ניהול המשאבים.

Director: סט של הוראות מחשב – שמגדיר פעולה רציפה שמורכבת מכמה וכמה פעולות. מדובר בראשית של מערכות ההפעלה. לפני שהופיע director הכל היה מורכב מהתערבות ידנית של מפעילים אנושיים – היה צריך להזין/לפקח/לקבוע דברים שונים. לאחר כניסת director (שנות 60) – היה שינוי גדול, היה אפשר להפעיל דברים הרבה יותר מורכבים שקודם בני אדם לא יכלו לפקח עליהם (או שהיה קשה לפקח עליהם).

Services שמציעה מערכת ההפעלה

ממשק משתמש – User interface (UI): כמעט לכל מערכת הפעלה יש UI (למערכות ישנות אין). לרוב נראה כמה וכמה סוגים בכל מערכת הפעלה. וישנם כמה סוגים שונים:



- **CLI: Command Line interface** – המשתמש מזין פקודה, ה-CLI לוקח את הפקודה (fetch) ומבצע אותה – הופך אותה לרצף של system calls.

כיצד מממשים CLI?

1. כחלק מ-kernel: תמיד נמצא בזיכרון, וה-command interrupter קופץ למקום המתאים בזיכרון.
2. לכתוב תוכניות, כך שכל תוכנית מממשת פקודה אחרת (ושם התוכנית יהיה זהה לשם הפקודה) ולשמור את התוכניות בדיסק. למשל – אם ב-Unix נבצע `rm file.txt` – המערכת תחפש קובץ בדיסק שקוראים לו `rm`: בתוך הקובץ היא תמצא תוכנית – תעשה loading לתוכנית לזיכרון, ותשתמש ב-`file.txt` כקלט לתוכנית שרצה.

מהו השיקול לבחירת המימוש? אם נשמור בדיסק – יהיה חפיפה בין קטעי קוד. הדילמה זהה לשקילות על וקטור לעומת Pooling. מהירות לעומת תחזוקה.

- לעיתים נמצא מס' interpreters במערכת הפעלה: ואז נקרא להם shell.
- **GUI:** ממשק משתמש גרפי, הוא נותן את המטאפורה של "שולחן עבודה", עכבר, קיצורי דרך (גרירה עם העכבר, משיכה, עבודה עם המקלדת וכו').

סיכום מערכות הפעלה | © גיא יער-און

רוב המערכות נמצא גם CLI וגם GUI – מדוע בעצם? ה-CLI הרבה יותר מהיר – ובאמצעות GUI צריך לבצע הרבה יותר פעולות על מנת לבצע את אותה פקודה פשוטה דרך ה-CLI. אך: ה-GUI הרבה יותר פשוט ואין סינטקס.

- ישנם עוד ממשקי משתמש רבים – למשל VUI: Voice User interface, ממשק משתמש מבוסס מגע, ממשק משתמש מבוסס מציאות מדומה – למשל VR Interface וכו'.

היכן עוד מסייעת מערכת ההפעלה למשתמש?

- הרצת תוכניות (טעינת תוכנית לזיכרון, הרצת התוכנית, סיום הרצה. כאשר התוכנית מסתיימת מעצמה, או כאשר התוכנית מסתיימת בצורה לא טבעית – יש להצביע על השגיאה ולטפל בה).
- טיפול ב-IO – נדרש קלט/פלט בתוכנית, נרצה שהתוכנית לא תגש ישירות כמובן – תמיד לפנות למערכת ההפעלה (הגנה על המשאב, ויעילות השימוש במשאב).
- טיפול בקבצים (נדון על כך בהמשך הקורס – קריאה מתוך קבצים, כתיבה לקבצים וכו').
- תקשורת בין תהליכים: העברת מידע בין תהליכי שפועלים באותו מחשב / שפועלים במערכות מרוחקות ומקושרים באמצעות רשת.
- טיפול בשגיאות – לא רק שגיאות תוכנה, אלא גם שגיאות חומרה (בעיה פיזית בזיכרון, שגיאה בהתקני IO: למשל נגמר הנייר במדפסת / נפלה הרשת וכו'). בכל סוג כזה של שגיאה – מערכת ההפעלה צריכה לצאת לפעולה על מנת לעזור לנו בצורה היעילה יותר.
- לרוב מספקת מערכת ההפעלה Debugging facilities שמקלות על התכנית להבין מה קרה ולדבג.

כיצד מערכת ההפעלה שומרת על יעילות המערכת?

- הקצאת משאבים: כאשר ישנם כמה וכמה משתמשים מחוברים במקביל / תהליכים שרצים במקביל – מנהלת בהתאם.
- Accounting: ניהול רישום של השימוש במשאבים השונים לפי משתמשים לאורך ציר זמן – כל פעם שתהליך של המשתמש צריך להשתמש באיזשהו משאב, נרשם בצד באיזה משאב השתמש המשתמש שלי, מתי, וכמה זמן הוא השתמש. מדוע צריך זאת? משתי סיבות: 1. Billing – כלומר לחייב בהתאם לשימוש 2. עבור סטטיסטיקות עתידיות, שישמשו את המעבד בצורה מושכלת יותר בעתיד (על מנת להכיר את המשתמש).
- הגנה ואבטחה על משאבים:

Protection: בכל פעם שניגשים למשאבים של המערכת – ניגשים בצורה שהיא תקינה, יש בקרה של המערכת. למשל: אם משתמש רגיל שאיננו מנהל מערכת ינסה למחוק קובץ מערכת.

Security: אימות משתמש, הגנה על משאבי IO מגישה לא חוקית או זדונית. למשל: כשפותחים מחשב, נדרשים להזין סיסמה/טביעת אצבע.

System calls

מנגנון שמאפשר לתוכניות שרצות על המחשב לבקש שירותים מסוימים ממערכת ההפעלה. למשל: open, read, write, fork, exit וכו'. נרצה שמימוש של פקודות אלו יהיה דרך מערכת ההפעלה. לרוב הם כתובות ב-high level language: C או C++.

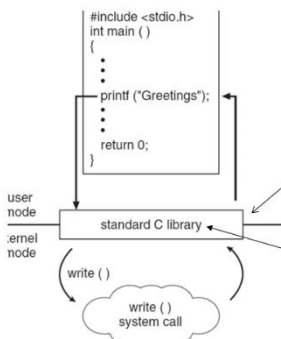
לכל system call יוצמד מספר, ותוחזק טבלה שמאנדקסת לפי מס' זה ואז בעת קריאה ל-system call נגיע לשורה זו בטבלה. בעת הקריאה, יפעיל system call interface את ה-system call המתאים ב-kernel ויחזיר את סטטוס הביצוע (ואת ערכי הפלט המתאימים).

דוגמה. המשתמש לוקח קובץ, ומעתיק את התוכן שלו לקובץ יעד. כמה system calls יתבצעו?

סיכום מערכות הפעלה | © גיא יער-און

תשובה. כרגיל, ה"כן, אבל" יהפוך ללא ניתן לדעת: בשלב ראשון, יש לשאול את המשתמש מיהם הקבצים של היעד והמקור. זה כבר דורש IO (הדפסה למסך), לאחר מכן צריך לבצע open, אם בכלל הקובץ קיים – אם לא: או שנסיים את התוכנית וזה גם system call, או שנחזור למשתמש ונבקש (IO) שוב מהמשתמש שם תקין. נניח והגענו למצב שצלחנו את זה – בידינו הקבצים הרלוונטיים: אנחנו נעבוד בלופ, נקרא קצת מהמקור ונכתוב ליעד, אך יתכן וגם בשלב זה נתקל בהרבה בעיות – אין מקום בדיסק, וכו'... נניח וגם את זה סיימנו – נצטרך לעשות close לקובץ, ולבסוף גם exit. מסקנה: יתכן שלפעולה כזו פשוטה נצטרך מאות עד אלפי system calls.

לרוב לא נקרא ישירות ל system call מהתוכנית שלנו, אלא נקרא לקריאה שנמצאת ב API (Application programming interface): מגדיר את סט הפונקציות שזמינות לאפליקציה שלנו ובכלל את הפרמטרים שנדרשים כקלט לפקודה והערכים שמוחזרים להם ניתן לצפות – והוא יקרא ל system call. כלומר: אנחנו לא צריכים לדעת את המימוש עצמו של system calls אלא רק לדעת מה נדרש ממני לשלוח. נחדד, כי אם היינו קוראים ישירות ל system call – היינו צריכים לדעת את המס' המדויק של הקריאה שיכול להשתנות בין מערכות הפעלה, את אופן העברת הפרמטרים, הפורמט המדויק של הפלט וכו'. לכן נרצה להשתמש ב API. יתרה מזאת – אם יש לנו שתי מערכות מחשב שונות שמשתמשות ב API: נצטרך רק להכיר סינטקטית API, זה מאפשר לנו להעביר קוד ממערכת למערכת ולקמפל אותו ולא יהיה בעיה. ללינוקס (Unix) יש את posix apin.



לדוגמה, משמאל: קוד C פשוט. בפועל, בקריאת printf מדובר בקריאה ל API של ספריית C, כעת ב API: הפונקציה printf תקח את הטקסט ותעבד אותו, תכין אותו ואת הפרמטרים הנדרשים להדפסה ובסוף תקרא ל write system call. נעיר כי API קורה כמו כן user mode.

```

#include <unistd.h> // עבור ספריית syscalls
#include <sys/syscall.h> // סכיל את המגדירות של ספריי הקריאות (SYS_*)

int main() {
    char* msg = "Hello\n";
    syscall(SYS_write, 1, msg, 6);
    return 0;
}
    
```

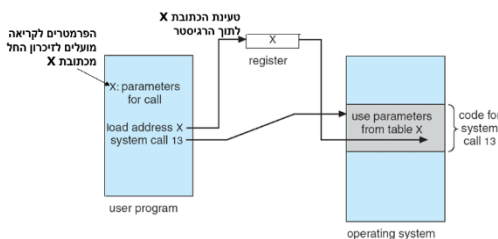
לצורך העניין, יכולנו גם לבצע את קריאת המערכת בעצמנו בצורה הבאה:

```
write(1, "Hello\n", 6);
```

או להשתמש גם בפונקציית המעטפת write:

כאשר קוראים ל system call לרוב צריך להעביר פרמטרים ואף סט של פרמטרים. ישנן 3 צורות כלליות להעברת פרמטרים למערכת ההפעלה:

1. העברה באמצעות רגיסטרים. אך – הרבה פעמים צריך להעביר יותר פרמטרים מכמות הרגיסטרים שיש לי.
2. שמירת הפרמטרים כבלוק בזיכרון, והעברת הכתובת של תחילת הבלוק באמצעות רגיסטר.
3. שימוש במחסנית: הפרמטרים יוכנסו למחסנית באמצעות התוכנית הרצה וימשכו ע"י מערכת ההפעלה. (pop)



מערכת MS-DOS

מערכת זו פותחה לפני 60 שנים, והיא מסוג single tasking: משתמש יחיד ופרוסס יחיד.

כאן, נבחין גם לפי התמונה כי בעת ש-process עלה לזיכרון, command interpreter קטן. מדוע? כי בעת שעלה תהליך מסוים – יש הרבה ב-command interpreter שהופך להיות לא רלוונטי, לכן אפשר להקטין חלק ממנו לטובת זיכרון עבור התהליך. בעת שהתהליך הנ"ל יסתיים, command interpreter יטען מחדש חזרה לגודלו.

עם זאת, עברו מהר מאוד למערכת שתומכת במס' משתמשים. ישנם הרבה אתגרים שנוספו כתוצאה מכך:

- אתגרי אבטחה – מניעת גישה לא מורשית למשאבי משתמשים אחרים.
- ניהול משאבים – קודם לכן היה משתמש יחיד, כעת יש כמה משתמשים ותהליכים שונים וצריך לחלק את המשאבים בצורה הוגנת.
- ניהול זיכרון מורכב הרבה יותר.
- הפרדה בין תהליכים על מנת למנוע התנגשות / הפרעה בין תהליכים.
- לשמור על יעילות הביצועים.

על אלו באה לענות UNIX:

כאן, כבר רצים multiprocessing, ואי אפשר להקטין את גודל command interpreter, שכן יתכן שתהליך B לא זקוק לחלק מהמידע שם, אך תהליך C כן צריך.

System programs

מדובר בתוכניות שהגיעו עם מערכת ההפעלה, ישנם המון סוגים שכאלו שמגיעים עם מערכת ההפעלה. כל מיני תוכניות שקשורות בניהול קבצים – סייר הקבצים למשל, ישנן תוכניות שנועדו לתת אינפורמציה ממערכת המחשב (למשל: שעון, לוח שנה וכו'), שינוי תוכן של קבצים (כמו notepad וכו'), קומפיילרים, דיבאגריים, browsers שמקבלים עם מערכת ההפעלה וכו'.

הרצאה 3

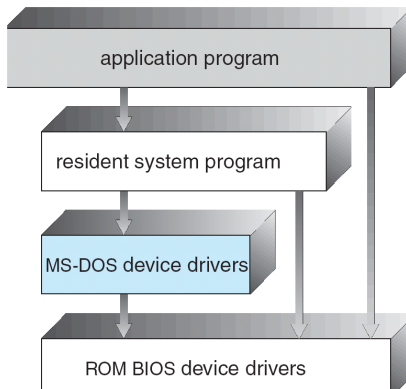
מימוש מערכות הפעלה

היכן נכתוב את הקוד של מערכת ההפעלה? הרי לבסוף, מערכת ההפעלה היא תוכנית.

- במקור, מערכות ההפעלה פותחו באסמבלי. בהמשך, עברו לשפות קצת יותר מתקדמות. כיום: רוב מערכות ההפעלה נכתבות ב-C++ או C. אך עדיין, ישנם חלקים במערכת ההפעלה שעדיין כתובים באסמבלי – החלקים שעדיין קרובים יותר לחומרה ודורשים את המהירות שמספקת אסמבלי. system programs (תוכניות שהגיעו עם מערכת ההפעלה) יהיו כתובות לרוב ב-C++.
- כאשר אנחנו באים לתכנן מערכת הפעלה – אנחנו צריכים לייצר ארכיטקטורה מסוימת. זו שאלה שאינה פתורה: יכולים להיות הרבה עיצובים למערכת ההפעלה. אבל – ישנם עקרונות כלליים שיכולים לעזור לנו שבאים לידי ביטוי בעיצוב של מערכות הפעלה, וכן יש עקרונות שנוגעים ספציפית בנושאים מסוימים שרלוונטיים למערכות מסוימות.
- **Batch**: פונקציונליות בה אנחנו מבקשים לבצע פעולה מסוימת (למשל: פעולת חיבור) אך לא מעוניינים לראות את התהליך עצמו שחישב אלא רק את התוצאה הסופית.
- **Time shared**: ההפך, כלומר כאן נרצה לראות את התהליך עצמו שחישב בנוסף לתוצאה הסופית.

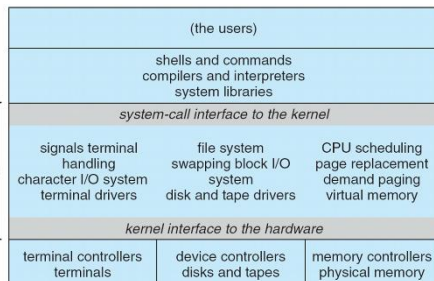
דוגמה ראשונה למבנה של מערכת הפעלה: MS-DOS

- מערכת מאוד ישנה, נוצרה בשביל לספק פונקציונליות מקסימלית במינימום משאבים. נועדה למשתמש יחיד, לא היה אפשר להריץ כמה תוכניות במקביל, נועד לשימוש ללא חיבור לרשת וכו'.
- הארכיטקטורה שלה משמאל. apd דיברו עם תחילתה של מערכת ההפעלה – והתוכנית הנ"ל העבירה את הדברים דרך device drivers לתוכנה עצמה.
- מה הבעיה בארכיטקטורה הזו? החיצים בין הרמות שלא סמוכות זו לזו. למשל, apd ל device drivers. שכן: היה אפשר לגשת ישירות לחומרה. הבעייתיות בכך, שיכולנו למחוק למשל מהדיסק את מערכת ההפעלה עצמה. אפשר להשתמש בחומרה בצורות שלא ייעודיות לכך מלכתחילה – ולנצל בצורה זדונית. ההנחה הייתה: שרוב הדברים שנעשים נעשים בצורה תמימה ונכונה – לאט לאט הופיעו איומים והבינו שעלינו להתמודד מול תוכנה זדונית.



דוגמה שנייה: Unix

- תוכנה לתמוך בכמה משתמשים שיריצו כמה תוכניות בו זמנית. כיצד נראית ארכיטקטורה זו? משמאל. עבדה ב"שכבות". בשכבה הראשונה – קומפיילרים, interpreters, בשכבה הבאה – kernel וכו'.



Layered approach: גישה למימוש מערכת הפעלה ב"שכבות". כל שכבה תלויה בפונקציונליות של השכבה לפנייה – כאשר שכבה 0 היא כבר פנייה ישירות לחומרה ושכבה N זו הממשק למשתמש. מה יתרונותיה של גישה זו?

סיכום מערכות הפעלה | © גיא יער-און

- Debug – ישנה הפרדה מלאה בין השכבות. ניתן להתחיל לדבג בשכבה מסוימת, ואם עדיין יש את הבאג הוא בהכרח בשכבות שמעליו. ניתן לצמצם את טווח החיפוש של הבעיה, ולמצוא מהר את הבעיה.

מה חסרונותיה?

- העיצוב עצמו, לרוב זה מסובך לחלק פונקציונאליות של מערכת הפעלה לשכבות מסוימות. שכן, יתכן שתוך עיצוב מערכת ההפעלה נגלה שאנחנו זקוקים לפונקציונליות משכבה מעל.
- יעילות – בכל פעם צריך לגשת לכל השכבות שלפניה.

Modules

רוב מערכות ההפעלה מפותחות בשיטה זו כיום, משתמשת בגישת תכנות מונחה עצמים. הרעיון הוא: במקום לבנות קרנל אחד ענק שכולל הכל מראש – או קרנל מינימליסטי שמעביר הכל למשתמשים, משתמשים במודולים:

- גישה דמוית תכנות מונחה עצמים – הקרנל מורכב מרכיבים נפרדים (כמו אובייקטים), ולכל רכיב יש תפקיד מוגדר וממשק ברור דרכו הוא מתקשר עם שאר המערכת.
- כל רכיב ליבה הוא יחידה עצמאית.
- טעינה דינמית: לא חייבים לטעון את כל הדרייברים כשהמחשב עולה, המודולים נטענים לזיכרון הליבה רק כאשר צריך אותם.

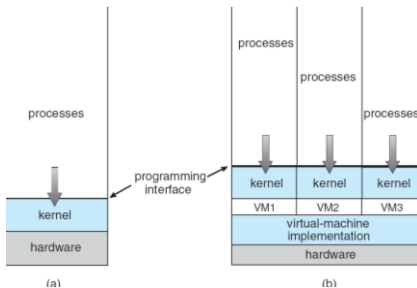
השוואה נחמדה: כשמקבלים טלפון אנחנו מקבלים את תוכניות המערכת עליו, ומתקנים בהתאם אפליקציות שנרצה. כך זה עובד בקרנל כאן – הליבה הבסיסית קטנה ומכילה רק מה שבאמת צריך, והמודולים הם קבצים נפרדים שיושבים בצד. אם למשל חיברנו מדפסת חדשה – הקרנל קורא למודול של המדפסת ומוסיף אותו לתוך עצמו בזמן אמת.

Virtual Machines

מדובר בתוכנית שתדמה מערכת הפעלה שיושבת מעל החומרה שלנו – למרות שהיא עצמה יושבת מעל מערכת הפעלה. הרעיון הוא, שבעת שנשתמש ב-Virtual machines נקבל ממשק שיושב מעל החומרה מתחת, ואז נוכל לכאורה להתקין מערכת הפעלה נוספת במחשב שלי.

בתמונה: דוגמה ל-virtual machine, מקבלים הפרדה מלאה.

- אנחנו רגילים לחשוב על שימוש בזה על מנת לקבל מערכת הפעלה אחרת במחשב שלנו. אך, ייתכן שנשתמש גם במחשב לינוקס ובכל זאת נוריד מכונה וירטואלית. מדוע? על מנת ליצור הפרדה בין המשתמשים – כל אחד יקבל מעין שרת משלו באמצעות המכונה הווירטואלית.

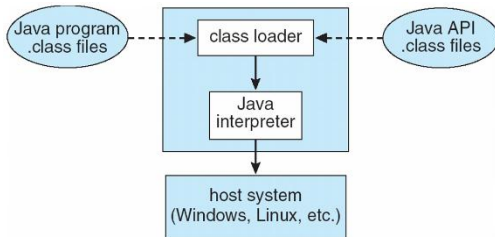


יתרונות שימוש:

- אפשר לבדוק את אותה אפליקציה שאנחנו מפתחים על אותה מערכת הפעלה מבלי שנצטרך לקנות מחשבים נוספים – לא צריך עוד חומרה. גם לא צריך להעלות מחדש את המחשב (שהרי יכלנו לבצע dual-boot ולהעלות למחשב כל פעם מערכת הפעלה אחרת).
- הפרדה מלאה בין כמה תהליכים שרצים.
- מונע פגיעה במערכת ההפעלה האמיתית – אם היה לנו תהליך שבגלל משהו זדוני שעשה הקריס לי את מערכת ההפעלה, עכשיו הוא לא יכול לעשות את זה. הוא במקסימום יפגע במכונה הווירטואלית ויגרום לי להעלות אותו מחדש.
- אפשר להריץ אפליקציות שונות שפותחו במקור למערכות הפעלה שונות ממה שיש לי על המחשב – מבלי לבצע ריסטארט של המחשב.

חסרונות שימוש:

- מייצרים עוד "שכבה" שצריך לעבור בדרך – וזה מאט את המהירות.
- לא נותן תאימות מלאה, תמיד ישנו מקום שלא נקבל את ההתנהגות המלאה של מערכת ההפעלה שנכתבה במקור.



JVM: המכונה הווירטואלית של ג'אווה. class loader מקבל קבצי class משני מקורות – הקוד שכתבת וספריות המובנות של ג'אווה. הקבצים הללו מקבלים bytecode שזהו קוד שרק ה-JVM מבינה ולא המעבד ישירות, interpreter לוקח את הbytecode ומעדכן אותו תוך כדי תנועה להוראות שמערכת ההפעלה יודעת להריץ. היתרון: בזכות המפרש לא צריך לשנות את הקוד שלנו.

System debugging:

Log files אלו הראיות הראשונות, המערכת כותבת כל הזמן מעין יומן אירועים – אם משהו לא עובד אבל המחשב לא קרס, הולכים ללוגים לראות מה השתבש בדרך. ברמת האפליקציה (core dump) מדובר כבר בקריסה של תוכנה אחת, מערכת ההפעלה מצלמת את הזיכרון שהיה שייך לאותה תוכנה ושומרת לקובץ, כדי שהמפתח יוכל לראות מה היה בתוך המשתנים רגע לפני הפיצוץ. ברמת המערכת (crash dump) זהו המקרה החמור ביותר – כל מערכת ההפעלה קרסה, כיוון שהכל נפל המערכת שומרת צילום של הקרנל כדי שיהיה אפשר להבין איזה דרייבר גרם לכל המחשב להיעצר.

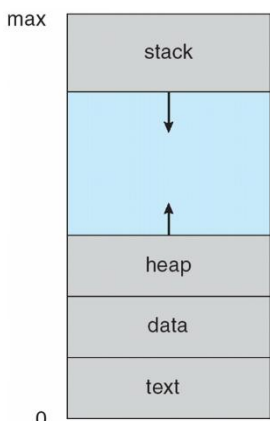
תהליכים – Process

בעבר – מערכות הפעלה אפשרו להריץ תהליך אחד (תוכנית אחת) בו זמנית. כיום: ניתן להריץ כמה בו זמנית.

הערה: אנחנו נתייחס לbatch systems וprocess jobs כמושגים זהים בקורס חופפים זה לזה. בעוד בפועל – jobs מתייחס לbatch systems וprocess מתייחס למערכות מסוג time-shared (מציג גם את הדרך, לא רק תוצאה סופית).

- איזה תהליכים רצים במערכת? תהליכים של מערכת ההפעלה, ותהליכים של המשתמש שמכילים user code.
- **Program:** מדובר ביישום פאסיבית (היא לא משתנה) ששוכנת על הדיסק, בעוד **Process:** ישות אקטיבית אשר משנה באופן רצוף את מצבה.

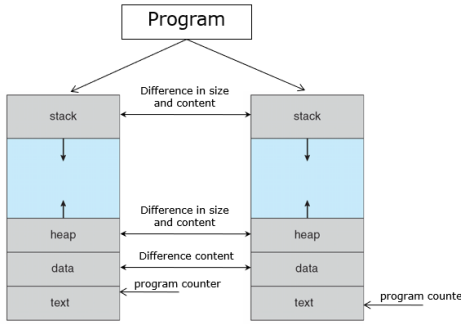
- הרצת התהליך מתבצעת בצורה סדרתית – זהו עקרון חשוב שכן בזכותו אנחנו תמיד יודעים איפה הפקודה הבאה שלנו.



- התהליך יושב ב"מרחב זיכרון וירטואלי" – הוא יושב בצורה לוגית מכתובת 0 לוגית (הכוונה היא לכתובת מדומה) עד לכתובת max לוגית. שוב: אלו לא הכתובות האמיתיות, אך מה שרלוונטי לנו הוא המבנה שנשמר בזיכרון ביחד.
- 1. Text נשמר הקוד.
- 2. Data נשמרים כל המשתנים הגלובליים – המשתנים שרצים עם התהליך לכל אורך חייו.
- 3. Heap ישן ההקצאות הדינמיות שעשינו.
- 4. Stack נשמרים משתנים לוקליים (כל פעם שקוראים לפונקציה, שיחיו במשך חיי הפונקציה, הם נוצרים במחסנית).

סיכום מערכות הפעלה | © גיא יער-און

על בסיס אותה תוכנית, ניתן להריץ שני תהליכים או יותר במקביל שלכל אחד יש execution sequence משלו. למשל: כאשר פותחים כמה וכמה קבצי word, לכל אחד מהם יש sequence שונה. משמאל – דוגמה:



נבחין:

- הText יהיה זהה בשני הsequence אך הprogram counter יהיה שונה שכן יתכן שאנחנו נמצאים במקומות שונים בקוד.
- הdata יהיה זהה בגודלו, אך שונה בתוכנו (אותם משתנים, אך יתכן שערכיהם שונים).
- הheap, stack יהיו שונים זה מזה. (לא בהכרח, אך לרוב).

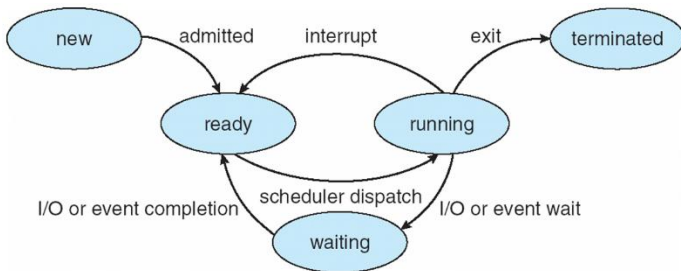
כל Process במהלך ריצתו משנה את המצב שלו (state). ישנם כמה מצבים:

- **New:** מדובר בתהליך שנמצא בתהליך יצירה, תהליך שעשינו לו loading. הוא עדיין "תינוק" ללא תעודת זהות, הוא רק מיוצר בשלב זה ולא ניתן להרצה.
- **Running:** כבר אפשר להריץ הוראות, אנחנו על המעבד.
- **Waiting:** מדובר במצב של המתנה, ממתינים לאירוע כלשהו שיקרה. למשל – IO של המשתמש שמעכב אותו.
- **Ready:** התהליך יכול להמשיך ולרוץ – אך אין לו כרגע את המעבד, לכן הוא ממתין.
- **Terminated:** התהליך קרס / הסתיים, וכעת מערכת ההפעלה צריכה לאסוף את המשאבים אחריו ולנקות אותו.

שאלה. כמה תהליכים יכולים להיות במערכת בכל רגע נתון בכל state?

תשובה. לא ניתן לדעת. הרי, בשביל שיהיו לנו כמה תהליכים בתוך new אנחנו זקוקים למס' המעבדים. לכן, הערך הזה קטן שווה ממס' המעבדים ואם נקצין: נאמר שזה 1 ומטה, שכן לא נרצה שיהיו לנו כמה תהליכים שנוצרים בזמנית. בדומה עבור Terminated ועבור running. כמה יהיה לנו waiting ובready? קטן שווה ממס' התהליכים במערכת.

כיצד תהליך עובר בין השלבים?

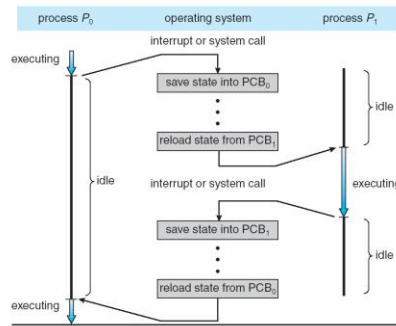


- מתחילים בnew, זה שלב ההיווצרות. בדומה: רק פעם אחת אנחנו terminated.
- משם, אנחנו עוברים אל מצב ready: תהליך זה נקרא **admitted** – נבחין שתהליך לא יכול לעבור ישירות אל running או waiting: שכן בשביל שיוכלו לבחור אותי בהרצה למצב running ויתנו לי את המעבד אנחנו צריכים להכנס למצב ready.
- ממצב ready ניתן לעבור למצב running אם הscheduler בחר בי להכנס (מיד נרחיב עליו). אנחנו למעשה יושבים ומחכים.
- ממצב running ניתן להגיע למצב terminated אם ביצענו exit, וניתן להגיע למצב waiting אם ביצענו IO למשל, וגם: ניתן לעבור למצב של ready – אם אנחנו למשל בtime shared והscheduler החליט שהיה לנו מספיק זמן – ואם למשל נכנס קוד של מערכת ההפעלה.
- ממצב של waiting ניתן להגיע אך ורק למצב ready: שכן חיכינו ועכשיו אנחנו מוכנים ומחכים לscheduler.

סיכום מערכות הפעלה | © גיא יער-און

process state
process number
program counter
registers
memory limits
list of open files
...

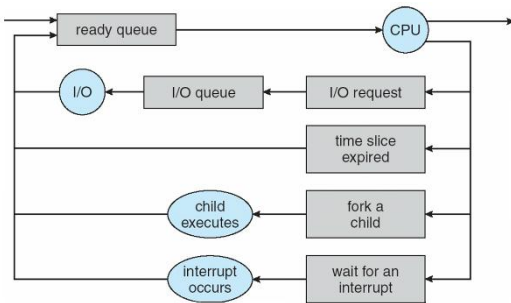
Process control Block (PCB): על מנת לתחזק את המעברים בין המצבים של התהליך, אנחנו שומרים מבנה נתונים בשם PCB – שם נשמור אינפורמציה כמו: מצב התהליך, מס' התהליך, PC (המקום בו אנחנו נמצאים בקוד), תוכן הרגיסטרים וכו'.



כיצד עוברים מתהליך לתהליך?

Process Scheduling Queues: המטרה שלנו היא שבכל עת יהיה תהליך שיכול לרוץ על המעבד (על מנת למקסם את ניצולת המעבד), לכן נשתמש ב-process scheduler: תורים בהם נעשה שימוש:

- Job queue: סט של כל התהליכים שבמערכת (לתור זה מגיעים כל התהליכים במערכת)
- Ready queue: סט של כל התהליכים אשר: 1. נמצאים בזכרון. 2. ממתנים להרצה.
- Device queue: סט של כל התהליכים שממתנים לIO.



במהלך הרצתו, התהליך עובר בין התורים השונים.

לדוגמה: תהליך נוצר ונכנס לתור ready. הוא יושב ומחכה – עד שהגיע תורו. הוא מבקש IO באמצעות system calls ונכנס לתור של IO, כשהIO שלו הסתיים הוא חוזר לתור של ready. מה עוד קורה לו בהמשך? Time slice expired: כשאנחנו חולקים את הזמן במעבד, נגמר הזמן שהוקצה לו, והוא חוזר שוב למצב של ready. יתכן ובהמשך – התהליך ייצור תהליך בן, ונרצה לבצע wait() ולעצור עד שהבן שלנו יסתיים, כעת נכנס למצב waiting. אפשרות נוספת – שאנחנו מחכים לinterrupt ולכן נמצאים בsleep.

Scheduler: ישנם שני schedulers עם מטרות שונות:

- **Long term scheduler:** בוחר התהליכים שיהיו ב-ready queue – בוחר מי יהיו בזיכרון. ה"יותר חכם", כיוון שאנחנו מפעילים אותו בתדירות יותר נמוכה ביחס ל-short-term scheduler, הוא "חושב קדימה" וההחלטות שלו מאוד משמעותיות והרות גורל. כל החלטה שלו – מעבירה תהליך מדיסק לזיכרון, לכן מופעל בתדירות די נמוכה. הוא שולט על מס' התהליכים שנמצאים בזיכרון בכל רגע (מושג זה נקרא **degree of multiprogramming** – דודי אמר לזכור מושג זה למבחן).
- **Short-Term scheduler:** בוחר את התהליך שירוצו על המעבד (מבין אלו שנמצאים ב-ready queue). מופעל כל כמה מילישניות, חייב להיות מאוד מהיר אחרת יהיה לנו overhead (תקורה – לא נרויח ממנו כלום – אך יתפוס לנו משאבים בזמן שלא הרצנו כלום). לכן ישתמש בשיטות די פשוטות להחלטתו כי הוא חייב לקבל החלטות בצורה די מהירה.
- **Overhead:** תקורה. מתייחס לכל משאב של המחשב (זמן מעבד, זיכרון וכו') שנצטרך לא כדי לבצע את המשימה עצמה אלא כדי לנהל אותה.

תהליכים מאופיינים כאחד מהשניים הבאים לרוב:

- **IO bound**: תהליך שרוב הזמן שלו הוא מבלה את זמנו בפעולות של IO – רוב הקוד שלו מריץ הוראות של IO. למשל: גלישה ברשת (הרי מדובר בהרבה הדפסה/קבלה מהמסך וכו'. החישובים קצרים) או העתקה של קבצים גדולים.
- **CPU bound**: תהליכים שרוב הזמן שלהם מריצים הוראות חישוביות – למשל חישוב של π מבצע הרבה חישובים – ואין שם יותר מדי הוראות IO.

חשוב: ה Long term scheduler צריך לדאוג שבכל רגע נתון יהיה בזיכרון שילוב של תהליכי IO, CPU. מדוע? כי אם יהיה לנו בזיכרון הרבה תהליכי IO התורים של IO יתפוצצו – והמעבד לא יהיה בשימוש, והנצילות תרד. מצד שני, אם יהיה לנו בזיכרון הרבה תהליכי CPU תור ready יתפוצץ ולא יהיה נצילות ושימוש ב IO.

Context Switch

הגדרה: כאשר המעבד עובר לטפל בתהליך אחר, המערכת חייבת לשמור את state של התהליך המסוים ו"לטעון" את state השמור של התהליך החדש – תהליך זה נקרא context switch.

- context switch עצמו שמור ב PCB (מבנה נתונים לתחזוק המצב הנוכחי).
 - זמן ביצוע context switch הוא overhead שכן המערכת איננה מבצעת עבודה שמשמשת את המשתמש בזמן זה – אלא משמשת את עצמה בהמשך. משך זמן הביצוע של context switch מושפע מהתמיכה בחומרה.
- שאלה לדוגמה (ממבחן).** תהי מערכת מחשב עם N תהליכים שחולקים CPU באמצעות מנגנון בשם RR (כל תהליך מקבל אותו זמן CPU). נניח כי כל context switch אורך S מילישניות וכל time quantum אורך Q מילישניות. מהו הערך המקסימלי של Q כך שהזמן המקסימלי בין זוג instructions של אותו תהליך הוא T מילישניות?

פתרון: למעשה אנחנו נשאלים כמה זמן (חסום ב T) עובר בין זמן של תהליך x לפעם הבאה שירוצץ במעבד. נבחין כי בין התהליכים ישנם:

- N פעמים context switch: בזמן כולל של SN
- N-1 פעמים שיתבצעו תהליכים אחרים: בזמן כולל של (N-1)Q

$$\text{לכן נדרוש: } SN + (N - 1)Q < T \text{ ונקבל: } Q < \frac{T - NS}{N - 1}$$

Process creation: כיצד יוצרים תהליך? משהו צריך להריץ פקודה בשם create על מנת להריץ פרויקט. לכן – לכל תהליך יהיה תהליך סבא שידאג ליצור אותו. למעשה אנחנו נקבל מעין עץ תהליכים, כאשר ראש העץ הוא scheduler שמנהל את התהליכים (אותו יקצה boot program). כל תהליך צריך לקבל משאבים/זיכרון וכו'. מי ייתן לו אותם? ישנם כמה גישות:

- האבא נותן לתהליך הבן את המשאבים שלו – שיחלקו אותם יחדיו.
- האבא מקצה חלק מהמשאבים שלו שיוקצו עבור הבן.
- האבא והבן לא יחלקו משאבים – לבן יהיה משאבים משלו (זה מסוכן לנו: שכן ייתכן שיהיה תהליך שייצור בכוונה הרבה תהליכים בנים על מנת לקבל עוד משאבים עבורו).

הרצאה 4

כפי שאמרנו בסיום ההרצאה הקודמת, תהליך נוצר ע"י מישור אחר. בלינוקס – זה ע"י פעולת `fork()` והBoot program יוצר את התהליך הראשון. ומה לגבי ההרצאה עצמה? תהליך האב יכול לרוץ במקביל לתהליך הבן – בזכות multiprocessing. אך לא תמיד הם ירצו לרוץ במקביל: למשל, אם נרצה ליצור תהליך בן שישאל את המשתמש כל מיני שאלות, עד שהמשתמש לא יענה – נרצה שבזמן הזה תהליך האב יחכה. בשביל זה ישנה פעולת `wait()`.

- מרחב הזיכרון (memory space) של הבן תהיה זהה לתמונת הזיכרון של האב בעת שנוצר תהליך הבן, אך הם יהיו נפרדים.
- ניתן גם לטעון תוכנית חדשה לחלוטין לתוך תהליך הבן, באמצעות פעולה `exec()`.
- כיצד מבדילים בין האב לבן? באמצעות ערך החזרה של פעולת `fork()`. ערך זה הוא `pid`. אם `pid < 0` אזי תהליך יצירת הבן נכשל. אם `pid = 0` אזי מדובר בתהליך הבן. אחרת, `pid > 0` ומדובר בתהליך האב.

Process termination

בסיום פעולתו של כל תהליך הוא נדרש להריץ את הפקודה `exit()` – מדובר בsystem call. במצב זה מערכת ההפעלה:

- מוחקת אותו מרשימת התהליכים הקיימים (job queue)
- כל המשאבים שהוקצו לתהליך יוחזרו למערכת ההפעלה, והיא תרשום שהמשאבים הללו אינם מוקצים לאף תהליך.
- אם התהליך הנ"ל היה תהליך בן, אזי אם הייתה בקשת `wait()` מהאב יוחזר לו כי סיימו.

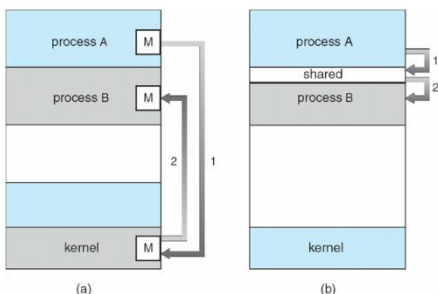
כמו כן, תהליך אב יכול לגרום להפסקת פעולת תהליך הבן באמצעות `abort`. מתי נרצה לעשות זאת?

- הבן משתמש ביותר מדי משאבים ממה שתכננו שהוא השתמש.
- המשימה שבשבילה נוצר הבן – כבר לא רלוונטית עוד. למשל: אנחנו סרבר, ויצרנו תהליך עבור בקשה מסוימת שהפכה ללא רלוונטית עוד. אז נרצה לבטל את התהליך הנ"ל.

Cascading termination: יש מערכות הפעלה שבעת שאבא ביצע `exit()` הורגות את כל העץ תחתיו – כל הבנים.

Interprocess Communication

תהליכים צריכים לשותף ולמצוא דרך לחלוק מידע בניהם. בין אם מדובר בתהליכים שרצים באותה מערכת או במערכות שונות. התמיכה בשיתוף המידע היא באמצעות IPC - interprocess communication. ישנם שני מודלים של IPC:



- **Shared memory:** מוקצה אזור בזיכרון הפיזי ששני התהליכים (או יותר) יכולים לגשת אליו. התהליכים קוראים וכותבים לאזור הזה כאילו הוא חלק ממרחב הכתובות הפרטי שלהם. זהו המנגנון המהיר ביותר כי הוא לא דורש התערבות של הליבה (Kernel) לאחר ההקמה, אך הוא דורש מהמתכנת לנהל סנכרון (כדי למנוע מצב ששני תהליכים כותבים בו-זמנית). בתמונה: מימין.
- **Message passing:** מערכת ההפעלה בונה מנגנון שמעביר "הודעות" בין התהליכים. היתרון שלו הוא שאין שום צורך לנהל דברים – זה תחום אחריותה של מערכת ההפעלה. החיסרון שלו הוא שהוא איטי יותר כי בגלל המעברים לkernel בכל הודעה הוא נדרש לעצור ולחכות. בתמונה: משמאל.

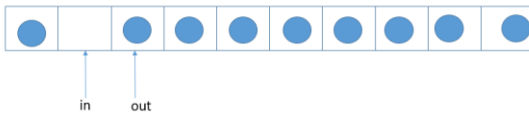
Producer-Consumer Problem: פרדיגמת היצרן-צרכן. מושג שנשתמש בו הרבה במערכות הפעלה. מודל קלאסי לשיתוף פעולה בין תהליכים, שבו תהליך אחד מייצר מידע ותהליך אחר משתמש בו. דוגמה נפוצה היא קומפיילר שמייצר קוד Assembly, אשר נצרך לאחר

מכאן על ידי ה-Assembler. כאשר משתמשים בפתרון של Shared Memory (זיכרון משותף) למימוש הפרדיגמה הזו, נהוג לדבר על שני סוגי חוצצים (Buffers) המשמשים להעברת המידע:

1. **Unbounded-Buffer (חוצץ ללא הגבלה):** במודל זה, אין הגבלה מעשית על גודל החוצץ (המקום אליו הם כותבים על מנת לתקשר בניהם) ולכן מבחינתם הוא אינסופי.

2. **Bounded-Buffer (חוצץ מוגבל):** מודל זה מניח שיש לחוצץ גודל קבוע ומוגדר מראש.

כיצד מממשים זאת? באמצעות שני מצביעים - in, out. במעין צורה ציקלית מעגלית. in הוא פוינטר שמצביע על התא הריק הבא שבו היצרן יכניס מידע חדש, out הוא התא שבו נמצא המידע הבא שהצרכן צריך לקרוא. ישנם שני מקרים:



- $in == out$: הצרכן יודע שאין לו מה לקרוא והוא צריך להמתין.
- $Out = (Buffer\ size) \% (in + 1)$: היצרן מתקדם ומגיע למצב שבו התא הבא שלו הוא out, המחסן מלא ולכן הוא צריך לחכות שהיצרן יוציא לפחות "תא אחד" ויתפנה מקום.

Message passing

- יוצרים משתנים משותפים ששני התהליכים מחזיקים.
- IFC Facility: מספק שתי פעולות, $send(message)$ ו- $receive(message)$. אם זוג תהליכים רוצים לתקשר בניהם עליהם ליצור communication link בניהם ולהעביר הודעות באמצעות הפקודות. מימוש ה-communication link הוא באמצעות חומרה.
- את תהליך ההודעות ניתן לחלק לשני אפשרויות: direct – בצורה ישירה. indirect – התהליכים לא שולחים הודעות אחד לשני אלא משתמשים במתווך שנקרא Mailbox (תיבת דואר). כלומר – במקום שתהליך א' יצטרך להכיר את השם של תהליך ב' – שניהם צריכים בסך הכל להכיר את תיבת הדואר. מערכת ההפעלה צריכה לספק פונקציות כדי לנהל את התיבות הללו כמו יצירה של תיבה, שליחה וקבלת הודעות והריסת תיבה.
- השוואה בין תקשורת ישירה לעקיפה:

Interprocess Communication –
Message Passing (cont.)

	Indirect Communication	Direct Communication
method	Primitives are defined as: $send(A, message)$ – send a message to mailbox A $receive(A, message)$ – receive a message from mailbox A	Processes must name each other explicitly: $send(P, message)$ – send a message to process P $receive(Q, message)$ – receive a message from process Q
Link established	only if processes share a common mailbox	established automatically
Link vs. process	link may be associated with many processes	link is associated with exactly one pair of communicating processes
Process pair vs. link	Each pair of processes may share several communication links	Between each pair there exists exactly one link
direction	Link may be unidirectional or bi-directional	The link may be unidirectional, but is usually bi-directional

Synchronization

המנגנון של message passing יכול להיות מאופיין כblocking or non-blocking: כלומר, האם השולח חוסם / לא חוסם את עצמו עד שההודעה מתקבלת. תצורת blocking נחשבת סינכרונית.

Blocking send: השולח מבצע block לעצמו עד שההודעה מתקבלת בצד השני.

Blocking receive: המקבל מבצע block לעצמו עד שההודעה זמינה לקבלה.

Non-blocking send: השולח שולח את ההודעה וממשיך בפעולתו – ללא חשיבות אם הצד השני קיבל אותה או שלא.

Non-blocking receive: המקבל מקבל את ההודעה גם אם היא invalid, הוא לא יושב ומחכה להודעה.

Buffering: המקום בו אנחנו משאירים את ההודעות ומחכים שיגיעו לצד השני. כמות המקום (ההודעות) שניתן לאחסן בו משתנה ויש כמה אפשרויות -

סיכום מערכות הפעלה | © גיא יער-און

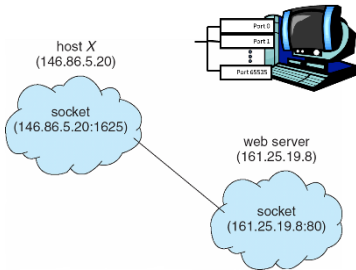
Zero capacity: לבאפר אין מקום להודעות שישבו ויחכו – בעת שליחת הודעה הצד השני צריך למשוך אותה, השולח חייב לבצע block לעצמו עד שהצד השני יקבל את ההודעה, הוא מחכה למקבל, המקבל מושך את ההודעה.

Bounded capacity: יש כמות סופית של הודעות שיכולות לשבת ולחכות, וכשהיה מלא אי אפשר לשלוח יותר הודעות.

Unbounded capacity: המקום בבאפר לא מוגבל – תמיד אפשר לשלוח הודעות. המצב הכי טוב לכאורה – מצריך הכי פחות ניהול.

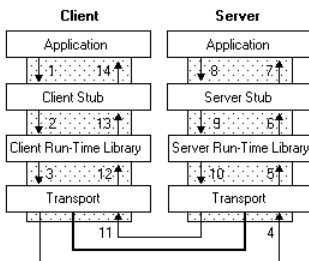
Socket – לא למבחן (לפי דודי)

עד כה, דיברנו על תקשורת בין תהליכים שהיו באותה מערכת הפעלה (אותו מחשב). ומה עם תקשורת בין תהליכים שנמצאים במחשבים שונים?



- לכל מכונה ברשת יש כתובת IP: X.X.X.X, שבאמצעותה ניתן לזהות את המחשב עצמו.
- זה לא מספיק לזהות את המחשב עצמו – לשם כך יש **port**: ערך מספרי שמייצג את האפליקציה בתוך המחשב. לצורך המחשה: יש בניין דירות, שמיוצג ע"י אותה כתובת, אך ה**port** הוא מספרי הדירה.
- בעת שנשויך את **port** ל**IP** ניצור **socket**. בעת ששני התהליכים מעוניינים לדבר – הם יצרו **connection**.
- ישנם שני פרוטוקולים שתפקידם להעביר הודעות בין אפליקציות שרצות על מחשבים שונים ברשת.
 1. UDP: "שולחים", לא אכפת לי מה יקרה" – טוב לשידורים חיים וכו'. לא מובטח שההודעה תשלח.
 2. TCP: אמין, מובטח שיגיע לנו הודעה.
- במהלך השנים התפתחו מוסכמות ל**port** מסוימים. למשל הוחלט כי למטרה מסוימת יוקצה **port** מסוים. ואז בשביל לבצע את אותה מטרה כל מה שצריך לדעת הוא את ה**IP**. למשל **PORT 80:HTTP**.

RPC – Remote procedure calls: לא למבחן (לפי דודי)



- מנגנון ה-RPC (Remote Procedure Call) נועד לגשר על הפער שנוצר כשעובדים עם **Sockets** – שבהם צריך לנהל דינאית את פתיחת הקשר, אריזת הנתונים (**Serialization**) ופענוח ההודעות. ה-RPC גורם לקריאה לפונקציה שנמצאת במחשב מרוחק להיראות ולנהל לקוד שלך בדיוק כמו קריאה לפונקציה מקומית. הוא "מסתיר" את כל הבלאגן של הרשת (**Sockets**, אריזת נתונים ושליחה) מאחורי קוד מתווך שנקרא **Stub**.
- התוכנה שלנו קוראת לפונקציה רגילה, קוד מקומי דרך תיווך (**stub**) אורז את הפרמטרים, תופס את הקריאה ושולח אותם לשרת. בצד השני: ה**server stub** פורק את החבילה, מפעיל את הפונקציה האמיתית בשרת ומחזיר את התוצאה באותה דרך.

RMI (Remote Method Invocation) הוא הגרסה של Java לעולם ה-RPC, והוא מאפשר לאובייקט שרץ ב-JVM (מכונה וירטואלית של ג'אווה) אחת להפעיל מתודות של אובייקט שנמצא ב-JVM אחרת, אפילו על מחשב שונה.

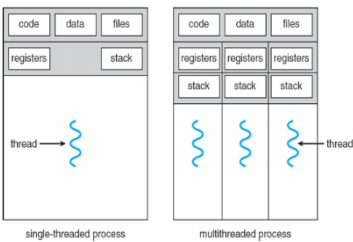
- **Busy waiting:** לרוץ בלולאה אינסופית ולבדוק תנאי – מבלי להמשיך. כאשר אנחנו במערכת עם מעבד יחיד – ביצוע **busy waiting** הוא חסר הגיון ואף מזיק. במערכת בעלת מעבד יחיד רק מעבד אחד יכול לרוץ בכל רגע נתון. אם תהליך א' מבצע **busy waiting** הוא תופס את המעבד ב-100%. הבעיה היא – תהליך א' מחכה לו יכול להשתנות רק ע"י תהליך ב' אבל כל עוד תהליך א' תקוע על המעבד בבדיקה שלו, תהליך ב' לא יכול לרוץ ולבצע את הפעולה שתשחרר את החסימה. המערכת תצטרך לחכות עד שיגמר זמן הריצה של א', ותבצע קונטקסט סוויץ' כדי שתהליך ב' יוכל לעבוד. בזמן הזה המעבד שרף זמן יקר.

Threads

עד כה הנחנו שכל תהליך מריץ תוכנית כך שבכל רגע נתון מתבצעת פקודה אחת בלבד של התהליך, אך ברוב האפליקציות המודרניות אנחנו רואים שימוש בmultithreading (שימוש בכמה חוטים בו זמנית, וכך מתבצעות כמה פקודות בכל רגע נתון).

לדוגמה: קליינט שולח בקשה לסרבר שמאזין לו, ובעת שמגיעה הבקשה הסרבר מייצר thread חדש שייתן את השירות לאותו קליינט וישוב להאזין לport שהיה בו. נשאל את השאלה: למה שלא ייצור תהליך חדש? מיד נבין זאת.

- threads רצים במסגרת התהליך והאפליקציה הבודדת.
- הרבה מהמשימות של אותו תהליך לרוב יכולים להיות ממומשים ע"י threads שונים. למשל: בעת שימוש בword נרצה שיהיה thread נפרד לטיפול בתצוגה, אחר לטקסט עצמו, תגובה להקשה על מקשים, הבאת מידע/שמירת מידע, וכו'.
- kernel של מערכת ההפעלה ממומש גם כן כמולטי-טרד.



הthread מורכב מ: thread id, program counter, register set, stack. עם זאת, thread חולק עם שאר threads של אותו תהליך את:

- Code section
- Data section
- משאבים אחרים של מערכת ההפעלה כמו קבצים וכו'.

יתרונות של threads:

אז למה שלא ליצור עוד תהליך במקום תרד?

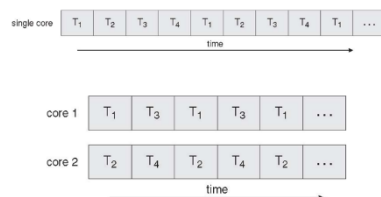
- הרבה יותר כלכלי ויעיל לבצע context switch לthread ולא לprocess שכן הם חולקים את משאבי התהליך. מדובר על פי 5 זמן בערך. כמו כן, הרבה יותר יעיל (פי 30) ליצור thread מאשר תהליך.
- מאפשר לתוכנית להמשיך לרוץ גם כאשר חלקים ממנה הם block (למשל יצאנו לIO) או שמבצע פעולה ארוכה (שהרי תרד רץ על חלקים שונים בתוכנית).
- threads חולקים את אותו הזיכרון והמשאבים של התהליך.
- אותו תהליך יכול לרוץ על מס' מעבדים במקביל באמצעות שימוש בthreads (כל אחד על מעבד שונה).

Multicore Programming

מערכות שהן multicore מביאות איתן אתגרים חדשים:

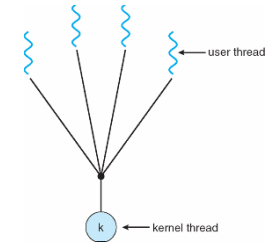
- קשה למצוא תחומים/ פעילויות שניתן לחלקם בצורה נקיה למשימות שיכולות להתקדם במקביל.
- חלוקת data ויצירת מבני נתונים מתאימים לתמיכה בהרצה במקביל.
- קושי משמעותי בtesting וdebugging. שכן הקוד הפך מדטרמיניסטי – לקוד multithread שכבר לא דטרמיניסטי – יתכן שהבאג קרה כי תרד אחד התקדם מהר מדי מתרד אחר: וכשנריץ שוב יתכן שזה כבר לא יקרה.

כיצד זה נראה בפועל? משמאל בתמונה. יהיו כמה ליבות (cores) שיעשו time-sharing. כשיש single core זו לא באמת ריצה במקביל לעומת המקרה שלמטה – שם זה ממש ריצה במקביל.

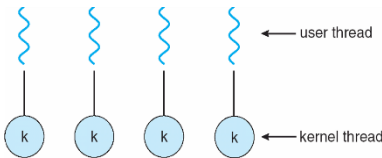


Kernel threads: התרדים שמכירה מערכת ההפעלה. כלומר, threads שמכיר scheduler. כשהמתזמן מחליט מי ירוץ עכשיו, הוא בוחר ישירות קרנל תרד.

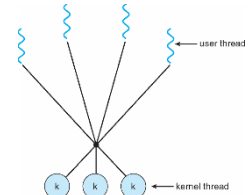
User threads: התרדים שמנוהלים באמצעות ספרייה (כמו pthread) בשם threading library ומשובצים ומנוהלים ע"י הספרייה עצמה. היא זו שמחליטה איזו תרדים כרגע רצים. מערכת ההפעלה לא יודעת שהם קיימים – היא רואה רק תהליך אחד גדול. הספרייה היא זו שמנהלת "לוח זמנים פנימי" ומחליטה מי מבניהם ירוץ עכשיו בתוך הזמן שהתהליך קיבל. היתרון הוא שהם מהירים מאוד, החיסרון הוא שאם תרד אחד מבצע פעולה חוסמת מערכת ההפעלה חוסמת את כל התהליך! כי היא לא מודעת באמת לכך שיש שם עוד תרדים בפנים. על מנת ש user threads באמת ירוצו על המעבד – הם חייבים "להתלבש" על קרנל תרד. ישנם כמה מודלים של מיפוי - המיפוי הוא הדרך שבה אנחנו מחברים בין העולם הווירטואלי של התוכנה לבין העולם הפיזי של המעבד. ישנם מס' מודלים:



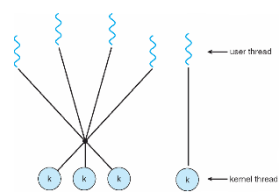
1. **Many-to-one:** מספר user-threads של התהליך ממופים כולם ל kernel thread אחד. מה היתרון של הדבר הזה? זה מאוד קל, וכל הניהול של threads ינוהל בספרייה עצמה (זה יעיל מבחינת משאבים ותקורה). החיסרון הגדול באמת זה שאם אחד התרדים מבצע פעולה חוסמת מערכת ההפעלה חוסמת את כל התהליך, ואז גם אם יש תרדים אחרים שיכולים להשתמש במעבד כרגע הם לא יכולים לרוץ כרגע כי המערכת חסומה. כמו כן: אין כאן באמת הרצה במקביל, גם אם למעבד יש כמה ליבות.



2. **One-to-one:** לכל user-thread ישובך kernel-thread. נקבל מקביליות אמיתית בהרצה, אין בעיות של חסימה וכו'. אבל: כמובן שיש חסרונות – זה יקר מאוד! ויש גם overhead שגורם לניהול של הקרנל תרדס ומביא הרבה מגבלות. יהיו הרבה תהליכים שלא יוכלו לרוץ בפועל כי נגיע למגבלה על כמות התרדים המקסימלית.



3. **Many-to-many:** במקרה זה ישנם m kernel threads וישנם n user threads. ומתקיים $m \leq n$. גם כאן פתרנו את החסימה.



4. **Two-level model:** במקרה זה, לחלק מה user threads שחשובים לנו אנחנו נעניק להם kernel thread ייחודי להם. ולשאר threads אנחנו נעבוד במודל many-to-many.

Java Threads

גם ב java ניתן ליצור threads. ניתן להרחיב את המחלקה Thread, או לממש ממשק בשם Runnable. המתודה start() מקצה זכרות ומאתחלת תרד חדש במכונה.

Threading Issues

יש המון בעיות שנוצרות מתרדים. נדון בכמה מהם:

- נניח וביצענו fork(). הוא צריך לשכפל רק את thread הקורא (זה שיצר את התהליך החדש) או את כל התהליך? העיצוב הנכון הוא בהתאם למטרת היצירה. אם מיד לאחר fork נבצע exec הרי כדאי לשכפל רק את התרד הקורא (נחסוך במשאבים ובזמן, הרי במילא נדרוס הכל). אחרת, נשכפל את הכל.
- נרצה לפעמים לסיים פעולתו של thread לפני שהוא סיים את פעילותו. למשל – אם מס' תרדים מחפשים בבסיס נתונים מידע ואחד מהם מצא את המידע. נרצה להפסיק את האחרים. ישנן שתי גישות עיקריות לעצירה:
 1. גישה אסינכרונית: "להרוג" את התרד. לא בהכרח נצליח לשחרר את המשאבים כמו שצריך.
 2. Deferred cancellation: מאפשר לתרד המטרה לבדוק באופן תקופתי האם עליו להתבטל – באמצעות בדיקה של דגל flag.

סיכום מערכות הפעלה | © גיא יער-און

- סינגלים מועברים לתהליך ונעשה שימוש בהם על מנת להעביר לתהליכים מידע לגבי אירועים רלוונטיים (למשל: חלוקה באפס או גישה לכתובת לא חוקית). מה הבעיה? לאיזה תרד נעביר את הסינגל הנ"ל? אפשר להעביר לכל התרדים של התהליך (ואז תקורה), אפשר להעביר לסט של תרדים שנקבעו מראש, אפשר להשתמש בתרד ייעודי שאליו יועברו כל הסינגלים, ואפשר להעביר לתרד שהסינגל הכי רלוונטי אליו (אך זה מחייב הבנה של לוגיקה).
 - Threads pool: הרעיון הוא יצירה של מס' תרדים מראש שנמצאים במאגר ומחכים למשימות. יש לכך יתרונות כמובן:
1. מהירות – מהיר יותר לטפל בבקשה לרוב באמצעות תרד קיים מאשר ליצור תרד חדש מאפס
 2. מאפשר להגביל את מס' התרדים באפליקציה לגודל המאגר הקבוע מראש. הסיכון ללא המאגר הוא שכמות בלתי מוגבלת של תרדים עלולה לכלות את כל משאבי המערכת.